Part IV

2D SLAM and the Cleaning Robot

Simultaneous Localization and Mapping (SLAM)

How to Build a Vision-Based Navigation and Localization Drone



Editor: Lee Shihyuan (李世淵)

LINE OA @prof

A Taiwan-Authored University Textbook

@prof

Simultaneous Localization and Mapping

Simultaneous Localization and Mapping, SLAM

Part IV 2D SLAM and the Cleaning Robot

Chapter 13

Building a 2D Map

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

Part III assembled all the tools: the coordinate transforms of Chapter 10, the probabilistic estimation of Chapter 11, and the optimization of Chapter 12. From this chapter, Part IV puts those tools together into a system that actually runs.

And the first thing to do is turn what the sensors measure into a map. The occupancy grid map of this chapter is the first map format in this book that can really be written to a file and handed to a navigation system—the SLAM systems of Chapters 14 and 15 output it, the cleaning robot of Chapter 16 plans its cleaning path on it, and Nav2 in Chapter 29 takes it as the standard input of the navigation stack.

This chapter makes one deliberate simplification: it assumes the pose is known. That of course does not hold—Chapter 1 called it a chicken-and-egg problem—but getting clear first on how to build a map with a known pose lets Chapters 14 and 15 concentrate on what to do when the pose is not accurate. This decomposition is pedagogically necessary.

Figure 13-0 How Chapter 13 Connects to the Rest of the Book

[Building on the previous chapters]

Chapter 10 Coordinate transforms → move LiDAR points from the body frame into the world frame

Chapter 11 Probability → each cell stores not 0 or 1 but an occupancy probability

Chapter 8 LiDAR specifications → angular resolution determines how finely the map

can be drawn

[This chapter is responsible for]

The data structure of the occupancy grid map → log-odds updating → ray casting

Scan matching (ICP) → fusing odometry with LiDAR

The resolution trade-off and evaluating map quality

[Where it leads]

→ Chapter 14 Gmapping: estimating the pose as well (the particle filter route)

→ Chapter 15 Cartographer: submaps + branch-and-bound loop closure (the optimization route)

→ Chapter 16 The cleaning robot: the consumer application of occupancy grid maps

→ Chapter 17 The three-dimensional version of ICP: point cloud registration

→ Chapter 29 Nav2 plans paths using this chapter's map as input

Learning Objectives

The first three parts laid the conceptual and mathematical foundations; from this part onward, theory comes down to earth on a real robot. Part IV focuses on the most approachable and most familiar form of SLAM—2D SLAM, exactly the technology a cleaning robot uses. This chapter first makes clear what a 2D map is and how it is built, preparing the way for Gmapping and Cartographer in the two chapters that follow. After completing this chapter, you should be able to:

1. Describe the structure of an occupancy grid map and the meaning of each cell's occupancy probability and how it is updated.
2. Explain how laser scan matching estimates a robot's relative motion.

3. Explain why odometry and LiDAR should be fused, and how the two complement each other.
4. Explain how the choice of map resolution affects map quality, memory, and computation.
5. Analyze the challenges of 2D mapping in typical indoor scenes such as corridors, classrooms, and offices.
6. Explain how to assess the quality of a 2D map and recognize common problems such as distortion, breaks, and drift.

Core Concepts

Chinese Term	English Term	Explanation
佔據格地圖	Occupancy Grid Map	Divides space into regular cells, each recording the probability of being occupied.
佔據機率	Occupancy Probability	The probability that a cell is occupied by an obstacle, between 0 and 1.
對數勝算比	Log-odds	Expresses a probability logarithmically, turning map updating into simple addition and subtraction.
掃描匹配	Scan Matching	Compares two successive laser scans to estimate the robot's relative motion.
ICP	Iterative Closest Point	The classic scan matching algorithm of repeatedly finding correspondences and aligning.
里程計	Odometry	Relative displacement estimated from wheel speed; continuous but drifting.
光線投射	Ray Casting	The method of updating the cells along a laser beam's path.

Chinese Term	English Term	Explanation
地圖解析度	Map Resolution	The actual side length each cell represents, 5 centimeters for example.
自由空間	Free Space	A region the laser has passed through and confirmed to be unobstructed.
未知區域	Unknown	A region no sensor has yet observed.
地圖漂移	Map Drift	Gradual displacement or deformation of the map caused by accumulated error.
逆感測器模型	Inverse Sensor Model	The model inferring the occupancy probability of each cell along the way from one measurement.

13.1 The Occupancy Grid Map

Chapters 1 and 3 gave a first acquaintance with the occupancy grid map; this section goes deeper into how it works—the central map representation of 2D SLAM, and what underlies the room map you see in a cleaning robot's app.

Dividing Space into a Checkerboard

The idea of the occupancy grid map is utterly plain: divide the plane the robot moves in into square cells (like a checkerboard or graph paper) and record in each cell one number—the probability that this cell is occupied by an obstacle.

- A probability near 1 (conventionally drawn black): almost certainly an obstacle, such as a wall or a table leg.
- A probability near 0 (drawn white): almost certainly empty and traversable.
- A probability of 0.5 (drawn gray): unknown—no sensor has yet seen this cell.

Distinguishing these three states (occupied, free, unknown) is crucial, and "unknown" above all—it is not the same as "free." The large gray expanse at the edge of a map marks places the robot has not explored, and treating it as traversable would make path planning assume it is empty and go wrong. This is where the occupancy grid map improves on a simple binary obstacle/no-obstacle map.

Each Cell Handled Independently

The occupancy grid map makes one key assumption: the occupancy states of the cells are mutually independent. This does not hold exactly in mathematics (adjacent cells are obviously related—a wall is continuous), but the simplification makes computation extremely efficient: each laser measurement needs only the few cells it struck to be updated, with no account taken of correlations among cells. This is another

instance of the engineering practice of trading a reasonable simplification for tractable computation, in the same spirit as the Gaussian assumption of Chapter 11.

The Premise: The Pose Must Be Known First

An idea that recurs from Chapter 1 must be stressed here: building an occupancy grid map presupposes knowing the robot's pose at every instant. For a laser measurement saying "there is a wall three meters ahead" can be assigned to a cell on the map only if one knows where the robot is and which way it faces.

Which raises the question—where does the pose come from? This is exactly SLAM's chicken-and-egg problem. This chapter assumes for now that the pose is supplied somehow (by odometry plus scan matching, say) and concentrates on how to build the map given a pose; how to estimate the pose well at the same time is the core of Gmapping and Cartographer in Chapters 14 and 15. Separating mapping from localization for the moment is an effective strategy for learning SLAM.

13.2 Updating the Occupancy Probability: Log-Odds

A robot does not draw its conclusion from one look—it scans the same place repeatedly, each scan supplying new evidence, and the map should accumulate that evidence. This section explains how the accumulation is done, and an elegant trick that makes the computation simple.

Accumulating Evidence: Bayesian Updating

The occupancy probability of each cell is updated with every observation. If a cell is struck by the laser many times in succession (each saying "there is something here"), its occupancy probability approaches 1; if the laser passes through it many times in succession (each saying "this is empty"), the probability approaches 0. This is exactly

the Bayesian updating of Chapter 11 in application—correcting an existing belief with new evidence.

But updating the probability directly requires repeated multiplication and division (and normalization), which is cumbersome, and the probability is confined between 0 and 1, where values near the boundaries easily become numerically unstable. Practice therefore adopts an elegant trick.

Log-Odds: Turning Multiplication into Addition

The trick is to rewrite the probability in log-odds form. The odds are the occupancy probability divided by the free probability, and taking the logarithm gives the log-odds. The benefits are immediate:

- Updating becomes addition: each new observation needs only its log-odds increment added—multiplication and division become addition and subtraction, and the computation is extremely fast.
- The numerical range is unbounded: log-odds may be any real number (positive infinity for certainly occupied, negative infinity for certainly free, 0 for an even chance), no longer pinned against the boundaries of 0 and 1, and numerically stable.
- Convert back to probability only when needed: accumulate in log-odds throughout and convert back to a probability between 0 and 1 only for display or decision-making.

This device of taking logarithms to turn multiplication into addition appears everywhere in engineering and science (the decibel and pH among them), and the occupancy grid map is only one instance. You need not memorize the formula, but you should understand the motivation: to make accumulating a great many observations

both fast and stable.

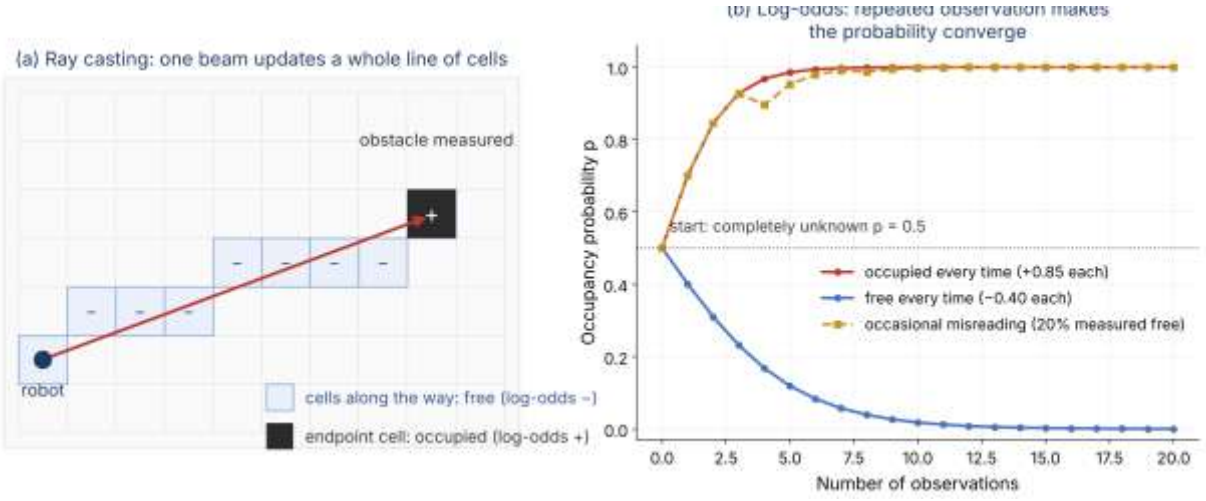
Ray Casting: One Measurement Updates a Whole Line

There is a detail here that is easily overlooked and very important. When the laser measures "a wall three meters ahead," it in fact updates a whole line of cells, not just the one at the wall:

- The cell at the laser's endpoint (the wall): raise the occupancy probability (there is something here).
- All the cells the laser passed through along the way (within three meters): lower the occupancy probability (the laser got through, so these places are empty).

This process of updating the cells along the laser's path is called ray casting, and the model behind it is the inverse sensor model—inferring the occupancy state of each cell along the way from one measurement. It is exactly this mechanism that lets the robot know not only where the walls are but also which space is definitely traversable, which is crucial for the path planning of Chapter 29.

Figure 13-1 Ray Casting and Log-Odds Updating



The left panel shows the geometry of ray casting. One laser beam measuring "something at three meters" in fact carries two kinds of information: there is an obstacle

in the endpoint cell (add a positive log-odds), and nothing blocked the way in each cell passed through (add a negative log-odds to each). So one beam updates not one cell but a whole line—which is why one circuit of a room is enough for the entire open area to be swept white.

The right panel explains why log-odds is used rather than storing the probability directly. The red curve is a cell measured as occupied every time: after the first observation the probability jumps from 0.5 to about 0.70, after three it exceeds 0.90, and after five it approaches 0.99. The blue curve is a cell measured as free every time, whose probability falls rapidly toward 0.

Most worth noting is the dashed yellow curve: this cell is measured as occupied most of the time, but with a 20% chance of being misread as free because of noise or a reflection. The curve dips only briefly and is pulled back—a single misreading cannot overturn accumulated evidence. This is the fundamental advantage of a probabilistic map over "mark it black when you see it," and the concrete realization of the Bayesian updating of Chapter 11 on a two-dimensional grid.

Why Take Logarithms

Bayesian updating is itself multiplication: new odds = old odds × measurement odds. Repeated multiplication readily underflows numerically, and it is slower.

After taking logarithms, multiplication becomes addition and each update needs only one integer or floating-point addition—a decisive difference for a real-time system updating tens of thousands of cells per second.

There is a hidden benefit as well: log-odds can be clamped directly (to ±6, say),

which keeps a cell observed a thousand times from locking up so that the map can never reflect environmental change. The dynamic updating of Section 13.5 relies on exactly this mechanism.

13.3 Laser Scan Matching

The previous section assumed the pose was known. But how is the pose actually estimated? Besides odometry, the LiDAR itself can supply a powerful cue to the pose—this is scan matching, the core technique of the 2D SLAM front end.

The Central Idea: Align Two Scans

The idea of scan matching is quite intuitive. The robot scans once at position A, advances a small step to position B, and scans again. If the environment has not changed (the wall is still that wall), then the displacement and rotation of the scan at B relative to the scan at A reflect the robot's motion from A to B.

In other words: find a rotation and translation that lay the scan at B onto the scan at A so that the two agree as closely as possible—that rotation and translation are the robot's relative motion. This is an optimization problem (in the spirit of Chapter 12): adjust the pose so as to minimize the discrepancy between the two scans.

ICP: The Most Classic Scan Matching

The best-known scan matching algorithm is ICP (iterative closest point). Its procedure is a loop of repeated approximation:

1. Correspondence: for each point in scan B, find the nearest point in scan A and pair them.

2. Alignment: compute a rotation and translation that minimize the total distance over all paired points.
3. Update and iterate: apply that transformation, then return to step 1 to find correspondences afresh, repeating until no further improvement occurs (convergence).

The name of ICP describes its operation precisely: iteratively find the closest points and align. It is simple and effective, the cornerstone of point cloud registration, and it reappears in the 3D mapping of Chapter 17.

The Strengths and Pitfalls of Scan Matching

- Strength: LiDAR is precise, and the relative motion estimated by scan matching is usually far more accurate than odometry, especially while turning, where it is unaffected by wheel slip.
- Pitfall one—a repetitive environment: in scenes that look the same everywhere, such as corridors and empty halls, scan matching cannot find a unique alignment (slide along the corridor and the scans look much the same), and the estimate goes wrong. This is the same class of problem as "the curse of the white wall" in Chapter 9: a lack of distinctive features.
- Pitfall two—the initial guess: ICP needs a good initial guess to converge to the right solution, or it may fall into a wrong local optimum. This is exactly where odometry comes in—the next section explains.

13.4 Fusing Odometry with LiDAR

Part II stressed sensor fusion repeatedly, and 2D SLAM is the cleanest demonstration of the idea: the strengths and weaknesses of odometry and LiDAR are exactly complementary, and together they far surpass either alone.

How the Two Complement Each Other

Characteristic	Odometry (wheel speed)	LiDAR scan matching
Data continuity	Continuous, high-rate, smooth	Limited by the scan rate
Short-term accuracy	Fair	High (especially while turning)
Long-term behavior	Accumulating drift	Comparatively stable
Where it fails	Slip, uneven floors	Repetitive environments, insufficient features
Initial guess required	Not required	Required (or it readily diverges)

Table 13-1 How Odometry and LiDAR Scan Matching Complement Each Other

The Logic of Fusion: Odometry Gives the Starting Point, LiDAR Corrects It

Do you see the key? Odometry needs no initial guess and is continuous and smooth, which is exactly the starting point that scan matching—which does need an initial guess—requires; and scan matching is precise and does not drift, which is exactly what corrects odometry's drift. The two fit together perfectly:

4. The robot moves one step and odometry first gives a coarse estimate of the relative motion (the initial guess).
5. Starting from that guess, scan matching (ICP) is run to refine a more accurate relative motion.
6. The refined pose is used to update the map (Section 13.2) and serves as the basis for the next step.

This cycle of "odometry predicts, LiDAR corrects" is the concrete embodiment of the predict-update cycle of Bayesian filtering in Chapter 11: odometry performs the prediction (the motion model) and LiDAR the update (the observation model). You should be able to feel the abstract mathematics of Part III turning here into an operable mapping procedure. Gmapping in Chapter 14 and Cartographer in Chapter 15 are both refinements of this basic cycle.

What to Do When the Wheels Slip

When the wheels slip, odometry suddenly reports a large, wrong displacement. The value of scan matching stands out all the more here: even if odometry says "I advanced half a meter," so long as the LiDAR scan shows the environment has barely changed, scan matching pulls the estimate back to "I hardly moved at all." This ability to overrule a wrong motion estimate with an observation of the environment is the fundamental reason SLAM is more robust than dead reckoning on odometry alone.

13.5 Map Resolution and Updating

In building a 2D map, one key decision must be made: how large an actual area does each cell represent? This is the map resolution, and it drives a three-way trade-off among map quality, memory, and computation.

The Resolution Trade-Off

Map resolution means the actual side length of each cell, commonly 5 centimeters indoors. The choice is a typical engineering trade-off:

- High resolution (small cells, 1 centimeter say): the map is fine and can represent small structures (table legs, door frames), but memory use grows explosively

(the number of cells varies inversely with the square of the side length—halve the side and the count quadruples) and computation slows.

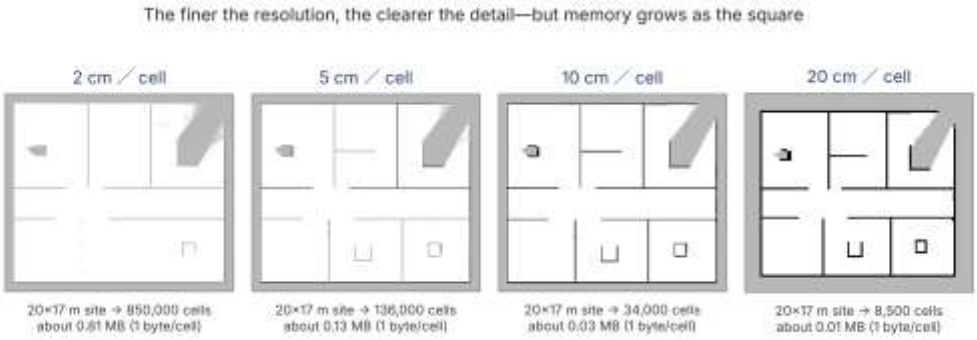
- Low resolution (large cells, 20 centimeters say): memory is saved and computation is fast, but the map is coarse, small obstacles may be smeared away within a cell, and wall positions are imprecise.

How to choose? It depends on the application. A cleaning robot need not resolve a centimeter of detail and 5 centimeters is ample; a robot requiring precise docking may need higher resolution. This echoes the discussion of LiDAR angular resolution in Chapter 8—however high the map resolution, it can never exceed the resolving power of the sensor itself, and the two must be matched.

The Reality of Memory

To give a concrete sense of memory use: a room 20 meters by 20 meters at 5-centimeter resolution is 400 × 400 = 160,000 cells. Raise the resolution to 1 centimeter and it becomes 2000 × 2000 = 4 million cells—fully 25 times as many. For a large area (a whole building) this growth is considerable, which is why Chapter 3 noted that 3D maps are compressed with an octree (OctoMap). Because a 2D occupancy grid covers only a plane, the memory pressure is far lower, which is one reason it suits teaching and cleaning robots.

Figure 13-2 The Resolution Trade-Off: One Site at Four Cell Sizes



These four panels are the same scan and the same data with only the cell size changed. At 2 centimeters the walls are finely drawn and the furniture outlines clear; at 20 centimeters the walls become thick black blocks, the table and chair legs blur together entirely, and even the doorway opening is nearly filled in.

But the price is written beneath each panel. For a site of 20 × 17 meters, 2 centimeters requires 850,000 cells and about 0.81 MB; 5 centimeters falls to 136,000; 10 centimeters leaves only 34,000; and 20 centimeters, 8,500. Halve the cell side and the count quadruples—memory grows as the square, and the computation needed for updating grows with it.

The common practical choice is 5 centimeters, and that is no accident: it is just fine enough to resolve an ordinary door frame (about 80 to 90 centimeters), a walkway (about 1 meter), and the gaps between table legs, while the memory remains within what an embedded platform can bear. Cleaning robots often use 5 to 10 centimeters to save memory; a warehouse AGV that must pass through narrow aisles may go to 2 or 3 centimeters.

The method of selection is exactly the same as judging LiDAR angular resolution in Section 8.4: ask first how narrow a gap you must resolve, then work back to the cell size. And the two must be matched—storing data from a LiDAR of 1-degree angular

resolution in 2-centimeter cells yields only a map full of holes, because the sensor simply cannot supply information that fine.

Dynamic Updating: A Map Is Not One Photograph

A map must be updated continuously, for two reasons:

- Exploring new areas: the robot reaches places it has not been and fills unknown gray cells in as free or occupied.
- Correcting old estimates: repeated observation of the same area accumulates evidence that corrects previously uncertain cells (the Bayesian updating of Section 13.2).

But this brings a subtle problem: what if the environment itself changes—if someone moves a chair? If the map's memory of old observations runs too deep (log-odds accumulated too high), it is hard to update to the new state; if the memory is too shallow, noise disturbs it easily. In practice the log-odds are clamped to upper and lower limits, which keeps the map somewhat plastic and able to reflect change. This foreshadows the discussion of dynamic environments for the cleaning robot in Chapter 16.

13.6 2D Mapping in Corridors, Classrooms, and Offices

With the theory covered, we turn to real indoor scenes. Different spatial structures pose quite different challenges for 2D mapping. Understanding them lets you anticipate where trouble will arise, which is important preparation before implementation.

The Classroom: A Friendly Scene for 2D SLAM

An ordinary classroom is the ideal scene for 2D SLAM: four walls with clear

boundaries, desks and chairs offering rich geometric features, and an enclosed space of modest size. Scan matching runs stably here and the map comes out clear. This is also why the hands-on activities in this book mostly use a classroom as the site (Chapters 14 and 29)—it lets students build a first experience of success under the most favorable conditions.

The Corridor: Scan Matching's Nightmare

A long straight corridor is the classic difficulty of 2D SLAM and the archetype of the repetitive-environment pitfall of Section 13.3. A corridor has parallel long walls on both sides, and moving along it the laser scan barely changes—scan matching cannot tell how far along the corridor the robot has traveled (that direction lacks constraint) and can determine only whether it has strayed from the centerline (the perpendicular direction is constrained by the walls).

This is exactly the source of that uncertainty ellipse stretched along the corridor in Section 11.2 of Chapter 11. Here the value of odometry stands out—it supplies the estimate of motion along the corridor and fills in the constraint scan matching lacks. This is one more instance of why fusing odometry and LiDAR (Section 13.4) is indispensable. Mapping a long corridor on LiDAR alone almost always fails.

The Office: Dynamics and Glass

An open-plan office combines several difficulties: glass partitions let the LiDAR see through or produce false reflections (the glass problem of Chapter 8), people moving about cause dynamic disturbance, and similar cubicle arrangements are easily confused. Such environments often require finer parameter tuning or additional sensors.

Lobbies and Atriums: The Difficulty of Open Space

The problem with large open spaces (a high-ceilinged lobby, say) is that LiDAR has a limited measurement range (about 8 to 12 meters for teaching models, Chapter 8), and when the space is larger than that range the robot in the middle may find no wall in any direction and lose its reference entirely. Scan matching then has nothing to work with and odometry alone must carry the load, with error accumulating rapidly.

Scene	Principal Challenge	Key Remedy
Classroom	Almost none (an ideal scene)	Map directly
Long corridor	Repetitive environment, no constraint along the corridor	Rely on odometry to supply the longitudinal estimate
Office	Glass, moving people, similar cubicles	Parameter tuning, multiple sensors
Large open hall	Beyond LiDAR range, nothing to scan all round	Carry on with odometry, extend the range, or add landmarks

Table 13-2 Mapping Challenges in Typical Indoor Scenes

13.7 Assessing Map Quality: Distortion, Breaks, and Drift

With the map built, how does one judge whether it was built well? This is an extremely important and often neglected part of practice. This section teaches you to recognize three common symptoms in a map and how to assess quality—the basis, too, of the project grading rubric of Chapter 31.

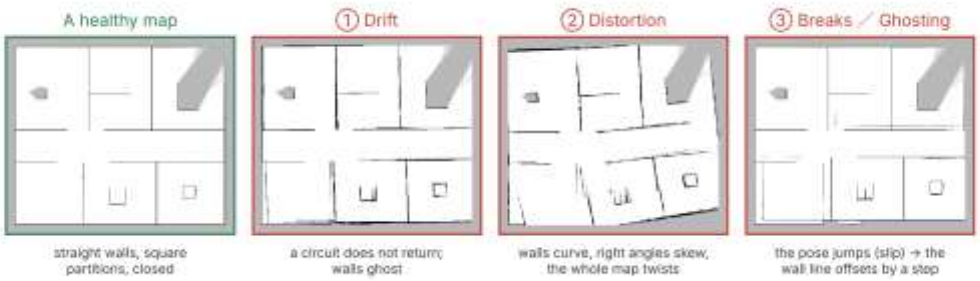
Three Common Symptoms

- Drift: the most common problem. Accumulated error gradually displaces the map, the classic symptom being that a circuit does not return to its starting

point—a loop that should close leaves the end offset from the beginning. This is exactly what the loop closure of Chapter 12 exists to fix, and the basis for judging whether loop closure detection is needed.

- Distortion: the geometry of the map is warped, straight walls become curved, right angles become skewed, and room proportions are falsified. This mostly comes from systematic error in scan matching or a scale error in odometry (an inaccurate wheel-radius calibration).
- Breaks and ghosting: the same wall appears twice on the map (a ghost), or wall lines break apart. Usually the pose estimate jumped wrongly at some point, so the same structure was drawn in two places. It happens most readily in corridors and open areas.

Figure 13-3 Three Common Symptoms in a Map



These four panels likewise come from one set of sensor data and differ only in the pose. The leftmost is how it should look with a correct pose: straight walls, square partitions, a closed outer boundary.

① Drift: the pose shifts slowly and persistently, so the same wall is drawn in different places at different times and finally appears as two parallel lines; and on returning to the same spot after a circuit, start and end do not line up. This is the classic symptom of no loop closure—the robot has come back and does not know it.

② Distortion: the pose carries a persistent angular bias and the whole map is progressively twisted. The symptom is that every short stretch looks normal while the whole is askew: the corridor is no longer straight and the room becomes a rhombus. The cause is usually an inaccurate wheel-radius or wheelbase calibration (actually calibrated in Chapter 29), or a failure of scan matching in a long corridor.

③ Breaks and ghosting: the pose jumps suddenly at some instant—the robot slips, is pushed by someone, or scan matching converges to a wrong solution. The symptom is easy to recognize: the wall is cleanly offset by a step at some position, as though the map had been cut and glued back without aligning.

Inferring the Cause from the Symptom

Seeing drift (ghosted walls, start and end not matching) → the loop closure was not detected, or was detected and not accepted. The branch and bound of Chapter 15 and the visual bag of words of Chapter 22 exist to solve exactly this.

Seeing distortion (curved walls, skewed right angles) → suspect a systematic odometry bias first (wheel radius or wheelbase), then degeneracy of scan matching in a long corridor.

Seeing breakage (a wall line cleanly offset by a step) → find what happened at that moment: slip, a collision, or dropped sensor frames.

This ability to read symptoms and infer causes will be in daily use while mapping on hardware in Chapter 29—and what it requires is precisely an eye that has seen what a normal map looks like.

How to Assess Map Quality

Assessment methods divide into the subjective and the objective:

- Visual inspection (qualitative): are the walls straight? Are the right angles square? Does the loop close? Is there ghosting or breakage? This is the quickest first step, and an experienced eye can tell a good map from a bad one at a glance.
- Comparison with measurement (quantitative): measure a few key distances on the map (corridor length, room width), compare them with tape measurements, and compute the percentage error. This is the objective way to quantify map accuracy.
- Loop consistency: have the robot go around a loop back to its starting point and measure the gap between "start" and "end" on the map. The smaller the gap, the better drift has been controlled.
- Comparison with ground truth (research grade): in an environment with a motion capture system or a precisely known map, overlay the estimated map on the truth and compare (used in the Chapter 31 project).

Developing the habit of assessing quality as soon as a map is built is crucial. A map that looks complete but is in fact distorted will throw all subsequent path planning (Chapter 29) off—the robot will drive into a wall following a wrong map. This echoes the idea of Chapter 8 that a sensor's limitation becomes the system's failure: map quality directly decides the success or failure of the whole navigation system.

By this point you understand what a 2D map is, how to build one, and how to assess it. But this chapter deliberately separated mapping from localization and assumed the pose was known. Real SLAM must solve both at once—and the next chapter brings in Gmapping, which uses the particle filter of Chapter 11 to let every particle carry its own map, localizing and mapping at once and untying the chicken-and-egg knot elegantly.

Theory and algorithm are at last about to become one.

Formulations and Procedures

1. Log-Odds Updating

For a cell with occupancy probability p, the log-odds are defined as:

$$l = \log(p / (1 - p))$$

With each new observation, the new log-odds require only an addition:

$$l_new = l_old + l_measurement$$

where l_measurement is the evidence this observation brings (positive for an occupied observation, negative for a free one). This is the mathematical core of "turning multiplication into addition" in Section 13.2. For display or decision-making, convert back to a probability:

$$p = 1 - 1 / (1 + \exp(l))$$

A positive l favors occupancy, a negative l favors free space, and zero is an even chance. In practice l is clamped to upper and lower limits to keep the map plastic to change.

2. The Optimization Objective of Scan Matching

Scan matching seeks a relative pose T (rotation plus translation) that minimizes the total error after aligning the current scan S to the reference scan R:

$$T^* = \operatorname{argmin} \Sigma_i \, \| T \cdot s_i - r_{j(i)} \|^2$$

where s_i is a point of the current scan and $r_{j(i)}$ its nearest corresponding point in the reference scan. This is a least-squares problem (Chapter 12), and ICP approaches its solution by repeating find correspondences—solve—iterate.

3. The Procedure for Mapping with Fused Odometry and LiDAR

Figure 13-4 One Step of 2D SLAM Mapping (this chapter's version, with loop closure handled separately)

① The robot moves one step

↓

② Odometry estimates a coarse relative motion (the initial guess)

↓

③ Starting from that guess, scan matching (ICP) refines the relative motion

↓

④ Update the robot pose (accumulate the refined relative motion)

↓

⑤ Ray casting: update the cells along the laser path in log-odds

↓

⑥ Return to ① for the next step (checking periodically for loop closure → Chapters 14 and 15)

4. Estimating Map Memory

The total number of cells in an occupancy grid map is:

$$number\ of\ cells = (width \,/\, resolution) \times (height \,/\, resolution)$$

For example, a 20 m × 20 m room at a resolution of 0.05 m: cells = 400 × 400 = 160,000. Halve the resolution (to 0.025 m) and the count quadruples (640,000)—this expression quantifies the quadratic relation between resolution and memory in Section

13.5.

13.8 Contemporary Practice in 2D Mapping: From a One-Off Map to a Lifelong Map

The mathematics of the occupancy grid map took shape in the 1980s, and the log-odds updating introduced in this chapter has not changed since. But how a map is used has changed considerably over the past decade, and that change bears directly on the tools you will use in Chapter 29.

1. From a Static Image File to a Continuously Updatable Pose Graph

The traditional practice is to map once, save a PGM image file plus a YAML configuration, and thereafter load that static map to localize while navigating. The procedure is simple but has an obvious problem—environments change. Move the desks and chairs, rearrange the shelving, redo the partitions, and the map is out of date with nothing for it but to map the whole place again.

slam_toolbox, the current mainstream tool in the ROS 2 ecosystem, proposes a different approach: it saves not just the map but the whole pose graph (the nodes and edges of Chapter 12), serialized. On the next start-up that pose graph is loaded, and the robot may choose to keep mapping, to localize only, or to merge new and old maps. This is called lifelong mapping.

A few specifications worth knowing: its synchronous mode runs at more than five times real time in a building of about 2,800 square meters (thirty thousand square feet), and still at three times at about 5,600 square meters; the largest known synchronous mapping case is about 18,600 square meters. The asynchronous mode suits larger spaces at the price of selectively discarding scans it cannot process in time. The

underlying optimization solver uses Ceres by default and can be switched to g2o—precisely those introduced in Section 12.6.

2. Localization Without a Static Map

slam_toolbox also offers a mode called elastic pose graph localization: after loading a serialized map, it matches incoming scans against a buffer of recent scans rather than permanently altering the underlying map. It likewise accepts an initial pose set in RViz and is used much like the AMCL to be introduced in Chapter 16.

The significance for students is that the AMCL of Chapter 16 (particle filter localization) is no longer the only option. The tool's own author nonetheless advises beginners to start with AMCL—because its behavior is intuitive, its documentation plentiful, and its problems easy to diagnose. This book's arrangement follows that advice.

3. Merging Several Maps

There is one more very useful capability in practice: merging the results of several mapping runs into one globally consistent map. A large site is hard to cover in one run—the battery runs out, some areas are temporarily inaccessible—so mapping in stages and merging afterward has become standard practice. Merging can be done by dragging into alignment by hand in RViz, or automatically with kinematic tools.

The mathematics behind this is the pose graph optimization of Chapter 12: join the two pose graphs, add a few manually specified edges saying "these two nodes are the same place," and solve again. Once Chapter 12 is understood, this feature is no longer mysterious.

4. Beyond the Occupancy Grid

Finally it is worth mentioning that the occupancy grid is not the only two-dimensional

map representation. Several others have gradually spread through navigation stacks in recent years: a costmap inflates obstacles outward and stores directly whether a place is traversable rather than whether something is there; a distance field (ESDF) stores in each cell the distance to the nearest obstacle, making obstacle avoidance and path smoothing more efficient to compute; and a semantic layer overlays information such as "this is glass," "these are stairs," "this is a keep-out zone" on top of the geometry—the layered costmap of Nav2 mentioned in Section 3.6 is exactly this architecture.

But nearly all of these representations are built on, or coexist with, the occupancy grid. So what this chapter teaches remains the foundation: once you understand that each cell stores a probability and accumulates evidence in log-odds, every variant is only another way of storing it.

Three Reminders for Students

1. Maps go out of date. Any robot meant to run for a long time must handle map updating rather than assume the environment never changes.

2. Saving the pose graph is worth more than saving an image file. An image can only be looked at; a pose graph can be optimized further, merged, and corrected in retrospect.

3. This chapter's simplification (a known pose) is lifted in the next. From Chapter 14, pose and map are estimated together—and that is what SLAM really looks like.

Case Studies

Case 13-1 That Map in the Cleaning Robot App: The Occupancy Grid in Daily Life

Open any cleaning robot app that supports mapping, and the room map you use to set keep-out zones and designate rooms is the occupancy grid map of this chapter underneath. The black wall lines are cells of high occupancy probability, the white cleanable area is free space of low occupancy probability, and places the robot has not visited are blank (unknown).

Watching this map grow is instructive: during the robot's first cleaning the map grows bit by bit—drawn where it goes and where it scans—and the unknown area is gradually filled in. This is the visualization of "dynamic updating—exploring new areas" in Section 13.5. If the robot slips somewhere or runs into black furniture (invisible to LiDAR, Chapter 8), you may see a skewed wall line or a gap in the map—the drift and breakage of Section 13.7, appearing live in your own living room.

A more advanced observation: some cleaning robots produce a more accurate and smoother map on a second cleaning of the same home. This is because more observations have accumulated and the estimate of each cell is more confident (the Bayesian accumulation of Section 13.2), and a loop closure correction (Chapter 12) may even have been performed. A cleaning robot is in fact a portable teaching aid for every theory in this part.

Case 13-2 Coming to Grief in a Long Corridor: A Real Teaching Failure

A scene played out almost every year in university classes: students run 2D SLAM mapping smoothly in the classroom, drive the robot confidently into a long corridor, and the map blows up instantly—the corridor drawn now long, now short, wall lines twisted and overlapping, the robot thoroughly lost.

The cause is the repetitive environment stressed in Sections 13.3 and 13.6: a long corridor lacks distinctive features along its length, and scan matching cannot determine how far along it the robot has traveled. If odometry is also inaccurate because the floor material changed (tile giving way to carpet), both sources of information fail and mapping must fail with them.

This failure is in fact the best lesson there is. It lets students feel for themselves that SLAM is not omnipotent and that the structure of the environment directly decides success or failure. The remedies echo the whole book: fuse dependable odometry (Section 13.4), use a longer-range LiDAR that can see the end of the corridor, or post AprilTags along it (Chapter 4) to supply distinctive landmarks. Understanding why it failed teaches more than a smooth mapping run in the classroom—which is why this book discusses limitations and failures honestly in every chapter.

Case 13-3 From 2D to Intelligence: The Evolution of the Cleaning Robot

To review the evolution of the cleaning robot is to review the spread of 2D SLAM. The earliest models (around 2002) had no map at all and used random bump-and-turn—a disorderly cleaning path, both repetitive and incomplete (Chapters 3 and 16). In 2010 Neato first fitted a 2D LiDAR to a cleaning robot, letting it build an occupancy grid map and plan a systematic path. In 2017 Xiaomi popularized the LiDAR cleaning robot at an affordable price (the case in Chapter 1), bringing the occupancy grid map into countless homes.

Recent models go further: a 3D ToF camera recognizes and avoids cables, shoes, and pet accidents (making up for the height blind spot of 2D LiDAR, Chapter 8); AI recognizes room types and zones automatically; and some even remember the maps

of several floors. This line of evolution shows clearly that the 2D occupancy grid map is the foundation, while real products keep layering on sensors and AI to make up for the limits of any single technology—exactly the fusion idea running through this book, and a preview of the AI semantic navigation of Part IX. The 2D mapping you learn today is the starting point for understanding all of it.

Hands-On Activities

Exercise 13-1 Building an Occupancy Grid Map by Hand

Purpose: to implement occupancy grid updating from scratch in code and so understand log-odds, ray casting, and scan matching first-hand. The exercise is mainly in Python, with a simulation route and a data route, and is a direct warm-up for the Gmapping exercise of Chapter 14.

Part One: Log-Odds Updating (everyone)

1. Create a two-dimensional array with numpy as the map (200×200 cells, say, each initialized to log-odds 0 for unknown).
2. Simulate a single laser beam: given the robot's position, the beam direction, and the measured range, implement ray casting—decrease the log-odds of the cells along the way (free) and increase that of the endpoint cell (occupied).
3. Convert the log-odds back to probabilities and display the map in grayscale with matplotlib (black = occupied, white = free, gray = unknown), confirming that one beam draws a white line to a black dot.
4. Simulate a full revolution (360 beams): place the robot at the center of the room, cast a beam toward each simulated wall surface, and observe whether the room outline is drawn.

5. Repeated observation: have the robot scan several times from the same position (with random noise added) and observe how the wall lines become more certain as observations accumulate (log-odds accumulate and the color darkens).

Part Two: Mapping While Moving (everyone, or undergraduate)

6. Design a simple simulated room (define a few walls in code), move a virtual robot along a planned route, update the map at each step from the known pose and the simulated LiDAR, and watch the complete room map grow step by step.
7. Add odometry noise: deliberately give the robot's pose estimate an accumulating error (drift) and observe how the resulting map deforms and the wall lines ghost—reproducing the symptoms of Section 13.7 first-hand.
8. Resolution experiment: map the same scene at resolutions of 2, 5, and 20 centimeters and compare map detail, memory use (cell count), and computation time.

Part Three: Mapping in ROS, Real or Simulated (for those with a ROS environment)

9. In the TurtleBot3 simulation (Chapter 28), launch the built-in 2D SLAM (slam_toolbox or gmapping), teleoperate the robot once around the simulated classroom, and watch the occupancy grid map grow in RViz in real time.
10. Drive the robot into a simulated long corridor and observe whether the corridor difficulty described in Section 13.6 appears (the map deforming along the corridor).

11. Once the map is built, assess its quality by the methods of Section 13.7: inspect visually whether the walls are straight, and measure the room dimensions on the map against the simulation settings.

Discussion Questions

1. In Part One, how does the color (occupancy probability) of the same wall differ after 1 observation and after 10? What does this show about the accumulation of log-odds?
2. After adding odometry drift in Part Two, which symptom appeared (distortion, breakage, or drift)? Which technique from Chapter 12 would you use to correct it?
3. Going from 5-centimeter to 2-centimeter resolution, by what factor did the cell count increase? Did the changes in memory and computation time match the quadratic relation of Section 13.5?
4. (With ROS) What went wrong with the long corridor map? Comparing with Section 13.6, which remedy do you think is most effective—adding odometry, using a longer-range LiDAR, or posting AprilTags? Why?

Extended Challenge (Undergraduate)

Implement a simplified ICP scan matcher: given two simulated scans with a known relative displacement, estimate that displacement by the procedure of finding nearest correspondences, solving for the best rotation and translation, and iterating, then compare with the truth. Then test your ICP in a long-corridor setting (too few features along its length) and observe why its estimate along the corridor goes wrong—which will give you a deep understanding of why Gmapping (Chapter 14) uses a particle filter rather than relying on scan matching alone.

Further Reading

1. S. Thrun, W. Burgard, and D. Fox, Probabilistic Robotics, MIT Press, 2005. (Chapter 9, Occupancy Grid Mapping, is the authoritative basis of this chapter.)
2. Cyrill Stachniss, the Occupancy Grid Maps, Scan Matching, and ICP videos in his SLAM course (YouTube). (Explains this chapter's core with animations; the first choice for self-study.)
3. P. Besl and N. McKay, "A Method for Registration of 3-D Shapes," IEEE TPAMI, 1992. (The original paper on ICP scan matching.)
4. A. Elfes, "Using Occupancy Grids for Mobile Robot Perception and Navigation," Computer, 1989. (The pioneering paper on occupancy grid maps.)
5. The slam_toolbox open-source project (GitHub) and the official ROS documentation. (The mainstream modern ROS 2 tool for 2D SLAM, superseding the old gmapping; the first choice for implementation.)
6. The SLAM chapters of the ROBOTIS TurtleBot3 e-Manual (emanual.robotis.com). (Complete implementation steps for 2D mapping.)
7. Xiang Gao and Tao Zhang, the map-related chapters of Fourteen Lectures on Visual SLAM. (Though focused on vision, its discussion of map representations complements this chapter.)

From a Known Pose to Estimating Both: Where This Chapter Stands

This chapter has completed the second half of SLAM: given the pose, turn sensor data into a map. That half is in fact quite direct—ray casting plus log-odds, and under a hundred lines of code will do it.

The difficult part is the other half. This chapter assumed throughout that the pose was known and correct, and Figure 13-3 has already shown how badly the map breaks once that assumption fails. A real robot has nobody to tell it where it is—it has only a drifting odometer and the scan matching of Section 13.3.

Scan matching is in fact already an embryo of SLAM: it corrects the pose in reverse by aligning the current scan to the map. But it has two weaknesses. First, it sees only locally—in the long corridor of Section 13.6 it can slide along and still match, so it cannot judge how far it has gone. Second, it has only one answer—once a match is wrong, there is no mechanism to go back.

Gmapping in Chapter 14 solves the first weakness with the particle filter of Chapter 11: it keeps dozens of pose hypotheses at once and lets the observations eliminate the implausible ones one by one. Cartographer in Chapter 15 solves the second with the pose graph optimization of Chapter 12: it keeps all history and corrects it in one pass at a loop closure. Read the two chapters together and you have a direct contest between the two routes of Chapters 11 and 12 on the same problem—judged by exactly the map-quality criteria taught in Section 13.7.

Summary

1. An occupancy grid map divides the plane into cells, each recording an occupancy probability in three states (occupied, free, unknown); "unknown" is not "free," which is where it improves on a binary map. Mapping presupposes a known pose, and this chapter separates mapping from localization to concentrate on how to map given a pose.

2. The occupancy probability accumulates by Bayesian updating with each observation; practice uses log-odds, which turns multiplication into addition and gives an unbounded, stable numerical range; one laser measurement updates a whole line through ray casting—the endpoint cell (occupied) and the cells along the way (free).
3. Scan matching estimates relative motion by comparing two successive laser scans, the classic algorithm being ICP (correspond—solve—iterate); it is precise but fears repetitive environments (corridors) and needs a good initial guess.
4. Odometry and LiDAR are exactly complementary: odometry is continuous and needs no initial guess but drifts, while LiDAR is precise and drift-free but needs an initial guess and fears repetitive environments; the logic of fusion is "odometry predicts, LiDAR corrects," the embodiment of the predict-update cycle of Bayesian filtering, and it can overrule a wrong motion estimate caused by slip with an observation of the environment.
5. Map resolution is a three-way trade-off among quality, memory, and computation, with the cell count varying inversely with the square of the resolution; the map must be updated dynamically to explore new areas and correct old estimates, and the log-odds are clamped to keep it plastic to change.
6. Different scenes pose different challenges: the classroom is ideal, a long corridor is hardest because it is unconstrained along its length (odometry must fill in), an office has glass and dynamics, and a large open hall exceeds the LiDAR's range; anticipating the challenge is the key to a successful implementation.

7. The three symptoms of a map are drift (a circuit does not return to its starting point), distortion (warped geometry), and breaks or ghosting (one wall drawn as two lines); assessment methods include visual inspection, comparison with measurement, loop consistency, and comparison with ground truth. Map quality directly decides the success of navigation.

This chapter deliberately separated mapping from localization and assumed a known pose. But real SLAM must untie both at once—exactly the chicken-and-egg problem of Chapter 1. The next chapter brings in Gmapping: using the particle filter of Chapter 11, it lets every particle carry its own trajectory and its own map and runs a survival of the fittest among many hypotheses, accomplishing localization and mapping at once and elegantly. The particle filter theory of Part III is about to show what it can do in the first complete SLAM system.

A Taiwan-Authored University Textbook

@prof

Simultaneous Localization and Mapping

Simultaneous Localization and Mapping, SLAM

Chapter 14

Gmapping: Particle Filter SLAM

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

Chapter 13 built a map, but it rested on an assumption that does not hold—that the pose was known. This chapter removes that assumption and confronts the chicken-and-egg problem posed in Chapter 1 head-on.

Gmapping's answer is the particle filter of Chapter 11, with one key twist: rather than letting the particles represent the joint distribution over pose and map, it lets each particle carry only one pose trajectory, with the map built by that particle itself by the method of Chapter 13. This decomposition is called Rao-Blackwellization and is the core of this chapter.

This chapter and Chapter 15 form a pair of controlled experiments: on the same problem (2D LiDAR SLAM), Gmapping takes the filtering route of Chapter 11 and Cartographer the optimization route of Chapter 12. Read both and then look again at the comparison of filtering and graph optimization in Section 12.7, and that table will no longer be an abstract list.

Figure 14-0 How Chapter 14 Connects to the Rest of the Book

[Building on the previous chapters]

Chapter 11 The particle filter → the skeleton of this chapter (predict, weight, resample)

Chapter 13 Occupancy grid mapping → each particle builds its own

Section 13.3 Scan matching → the key component for improving the proposal distribution in this chapter

[This chapter is responsible for]

Rao-Blackwellization: splitting the joint distribution into "particles for the pose, analytic for

the map"

Particle count, weights, resampling, and particle degeneracy

Gmapping's two key improvements: an improved proposal distribution and selective resampling

[Where it leads]

→ Chapter 15 Cartographer: the optimization solution to the same problem (the control group)

→ Chapter 16 AMCL: the particle filter applied to localization in a known map

→ Section 6.7 Automatic magnetometer calibration: the same RBPF applied to a different object

→ Chapter 29 Tuning Gmapping parameters on a physical vehicle

Learning Objectives

In the previous chapter we built a map assuming the pose was known, but real SLAM must untie localization and mapping at once. This chapter introduces the first complete SLAM system—Gmapping. Using the particle filter of Chapter 11, it lets every particle carry its own trajectory and its own map and runs a survival of the fittest among many hypotheses, untying the chicken-and-egg knot of Chapter 1 elegantly. After completing this chapter, you should be able to:

1. Describe the basic ideas of Gmapping and its place in the history of SLAM.
2. Explain the central idea of the Rao-Blackwellized particle filter: why letting each particle carry a map unties the chicken-and-egg problem.
3. Explain the roles of particle count, weight, and resampling, and the concepts of particle degeneracy and effective sample size.

4. Explain Gmapping's two key improvements: the improved proposal distribution and selective resampling.
5. Describe the complete procedure for running Gmapping in ROS and the prerequisites it requires.
6. Recognize common mapping failures—wheel slip, repetitive scenes, LiDAR occlusion—and propose remedies.
7. Complete the exercise of building a classroom map with a teaching vehicle or a simulator.

Core Concepts

Chinese Term	English Term	Explanation
粒子濾波 SLAM	RBPF-SLAM	A SLAM method estimating trajectory and map together with a particle filter.
Rao-Blackwell 化	Rao-Blackwellization	Decomposing the problem so that one part is sampled with particles and the other solved analytically.
粒子	Particle	One complete hypothesis: a trajectory together with its corresponding map.
權重	Weight	How well that particle's hypothesis agrees with the sensor observations.
重採樣	Resampling	The process of copying high-scoring particles and eliminating low-scoring ones by weight.
粒子退化	Particle Depletion	The loss of particle diversity caused by excessive resampling.
有效粒子數	Effective Sample Size	A measure of particle diversity, used to decide whether to resample.

Chinese Term	English Term	Explanation
提議分布	Proposal Distribution	The probability distribution generating candidate positions for the particles at the next step.
掃描匹配	Scan Matching	Comparing laser scans to refine the pose; Gmapping uses it to improve the proposal distribution.
選擇性重採樣	Selective Resampling	Resampling only when the effective sample size is too low, to avoid depletion.
地圖伺服器	map_server	The ROS tool for saving and loading map files.
地圖檔	pgm / yaml	The image and description file formats of a ROS map.

14.1 The Basic Ideas of Gmapping

Gmapping is one of the best-known and most widely used open-source systems in 2D laser SLAM. It was proposed by Grisetti, Stachniss, and Burgard in 2007—the German Freiburg and Bonn probabilistic robotics group introduced in Chapter 2. After finishing this chapter, watching Professor Stachniss explain particle filter SLAM himself on YouTube gives the rare experience of hearing it from the original author.

What Problem Gmapping Solves

Chapter 13 deliberately assumed a known pose and concentrated on mapping. But a real robot's pose is exactly what is unknown—and if the pose is wrong the map is wrong with it. Gmapping must answer two mutually dependent questions at once: what was the trajectory the robot traveled? And what does the map of the environment look like?

Its answer is most ingenious: do not try to find the one correct trajectory, but maintain dozens to hundreds of possible trajectory hypotheses at once, each building its own map; then see which hypotheses agree best with the sensor observations, let those survive and multiply, and eliminate the rest. This is the direct application of the survival-of-the-fittest idea of the particle filter in Chapter 11.

Inputs and Outputs

- Inputs: 2D laser scans (LaserScan, Chapter 8), odometry (Chapters 3 and 13), and the robot's coordinate frame configuration (TF, Chapter 10).
- Outputs: an occupancy grid map (Chapter 13) and the robot's pose in that map (that is, the map→odom transform, Section 10.7).

Note this list of inputs and outputs: it draws on almost every preceding chapter.

That is exactly the point of this part—theory converges here into a system that can really be run.

Its Place in History

Gmapping was for years the default choice for 2D SLAM in the ROS ecosystem, and countless university laboratories and teaching platforms began with it. In recent ROS 2 environments, graph-optimization solutions such as slam_toolbox (Chapter 12) have gradually become mainstream, but Gmapping's teaching value is undiminished: it is the clearest example of how a particle filter does SLAM, and the best bridge from the theory of Chapter 11 to a real system.

14.2 The Rao-Blackwellized Particle Filter

This is the central idea of the chapter. Gmapping's full name is Rao-Blackwellized particle filter SLAM, and behind that mouthful lies a very clever decomposition.

The Difficulty of Applying a Particle Filter to SLAM Directly

Recall Chapter 11: a particle filter approximates a probability distribution with a set of samples. Applied to SLAM directly, every particle would have to represent one hypothesis of both the robot's pose and the entire map. The trouble is that a map has tens of thousands of cells—tens of thousands of unknowns. Sampling such a high-dimensional space with particles would need an astronomical number of them, which is entirely impracticable. This is called the curse of dimensionality.

Rao-Blackwellization: A Clever Decomposition

The central insight of Rao-Blackwellization is to split the problem in two: sample one part with particles and compute the other analytically.

The key observation is the conclusion of Chapter 13: given the robot's complete trajectory, building the map is a deterministic and simple matter (log-odds updating will do, with no sampling required). In other words, the map need not be guessed at all; it can be computed directly from the trajectory.

The decomposition follows naturally:

- The part sampled with particles: the robot's trajectory (only three numbers per step: x, y, θ). This is low-dimensional and well within a particle filter's reach.
- The part computed analytically: the map. Each particle takes its own trajectory and computes the corresponding map directly by the method of Chapter 13.

The Core Idea: Every Particle Is a Parallel World

Imagine 100 parallel worlds, in each of which the robot walked a slightly different route (a trajectory hypothesis),

and so each drew a slightly different map.

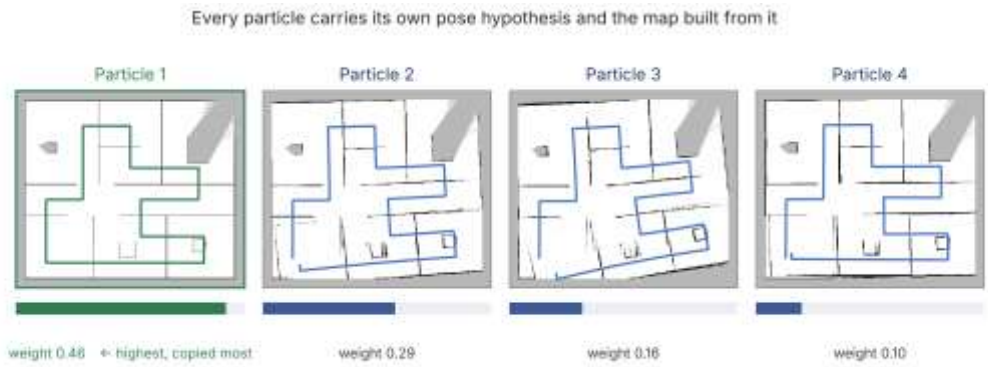
When a new laser scan arrives, every world asks itself: given my map and my position, is this scan plausible?

Plausible worlds score high (high weight); implausible ones score low.

At resampling, high-scoring worlds are copied and low-scoring ones vanish. What survives is the hypothesis most consistent with the observations.

That is Gmapping: finding the correct trajectory and map at once through competition among parallel worlds.

Figure 14-1 Rao-Blackwellization: Every Particle Carries Its Own Map



These four panels were produced from the same LiDAR data and differ only in the trajectory each of four particles hypothesizes—each particle has a different odometry error model, so each walks a different path and builds a different map.

Particle 1's hypothesis is closest to the truth, and its map has straight walls and square partitions; particle 4's hypothesis deviates most, and its map is twisted throughout with blurred walls. The weight is computed from exactly how well this laser scan agrees with my map—weights of 0.46, 0.29, 0.16, and 0.10, in exactly the order of map quality.

This is the whole spirit of Rao-Blackwellization. A particle need not guess pose and map at once (a dimension far too high to handle); it need only guess a trajectory, and the map is the inevitable consequence, computed by the method of Chapter 13. The particle filter thus goes from impossible to feasible.

Note one thing in particular: what is eliminated is not only the wrong pose but the wrong map with it. At resampling, particle 1 is copied several times and particle 4 deleted, and its skewed map vanishes along with it. This is the greatest difference between SLAM and localization alone—a wrong hypothesis contaminates pose and map together, so the whole set must be eliminated together.

A Frequently Asked Question: Is There Enough Memory?

Every particle carrying a map sounds memory-hungry—and it is. For 30 particles, a 20 × 17 meter site, and 5-centimeter resolution, the total is 30 × 136,000 = 4.08 million cells, about 4 MB.

That is nothing on a laptop, but on an embedded platform with only tens of MB of memory it is a real limit. It is also why Gmapping's particle count is usually set in the dozens rather than the thousands.

There is one implementation trick: at resampling, copied particles may share the same map (copy-on-write, duplicated only when written to), which saves a great deal of memory.

Why This Unties the Chicken-and-Egg Problem

The chicken-and-egg problem of Chapter 1 is that mapping needs a pose and localization needs a map. The RBPF's solution is that neither need come first. Each particle first hypothesizes a trajectory (which needs no map), builds a map from that trajectory (which needs only the trajectory), then evaluates how good the hypothesis is using the observations (which needs a map—and the map has just been built). The whole cycle is self-consistent and requires neither side to exist first. This elegant decomposition is Gmapping's important contribution to the history of SLAM.

14.3 Particle Count, Weights, and Resampling

This section goes into three practical essentials of the RBPF: how many particles to use, how to score them, and when to eliminate them.

1. Particle Count: Accuracy Against Computation

The particle count is Gmapping's most important parameter, and the trade-off is intuitive:

- Many particles (100, say): more hypotheses are covered and the system is more robust to complex environments and greater uncertainty, but every particle must maintain a map, so memory and computation cost more.
- Few particles (5 to 30, say): fast and light, but hypothesis coverage is insufficient, and in the face of large drift or a repetitive environment every particle may guess wrong with no way back.

In teaching practice, 30 particles usually suffice for an ordinary classroom or corridor; in a large environment or with poor odometry it can be raised to 50 or 100. One important reminder here: particle count times map size determines memory use—100 particles each carrying a 400×400 map is 16 million cells. This is the fundamental reason Gmapping is hard to use on very large sites, and one motivation for the later ascendancy of graph optimization methods (Cartographer, Chapter 15).

2. Weights: Scoring Hypotheses with Observations

A particle's weight represents how well this hypothesis agrees with the latest observation. It is computed by using that particle's pose and map to predict what should be measured and comparing with the actual laser scan.

If a particle's map says "a wall three meters ahead" and the actual scan also measures three meters, the two agree and the weight is high; if a particle's map says "ahead is empty" while a wall is actually measured, that particle's pose hypothesis is clearly off and its weight is low. This is the concrete realization of the observation model of Chapter 11.

3. Resampling: Survival of the Fittest, and Its Pitfall

Resampling copies high-scoring particles and eliminates low-scoring ones by weight, so that the particle set gradually gathers around the correct hypothesis. But a dangerous pitfall lurks here: particle depletion.

If resampling occurs at every step, the diversity of the particles is lost rapidly—in the end all of them may be copies of a single ancestor and nearly identical. The particle filter then loses the point of maintaining several hypotheses. Worse: if that heavily copied ancestor in fact guessed wrong, the system has no other hypothesis left to save it and SLAM fails completely.

This problem is especially severe in SLAM, because once a wrong trajectory hypothesis monopolizes the set, the corresponding wrong map is settled too and cannot be corrected afterward (the weakness of filtering relative to graph optimization noted in Chapter 12). Gmapping's second key improvement exists precisely to counter it—see the next section.

14.4 Gmapping's Two Key Improvements

A particle filter with one map per particle and nothing more would be impractical, because it would require a great many particles. Gmapping succeeds thanks to two improvements that cut the required particle count sharply. Understand these two and you understand the essence of Gmapping.

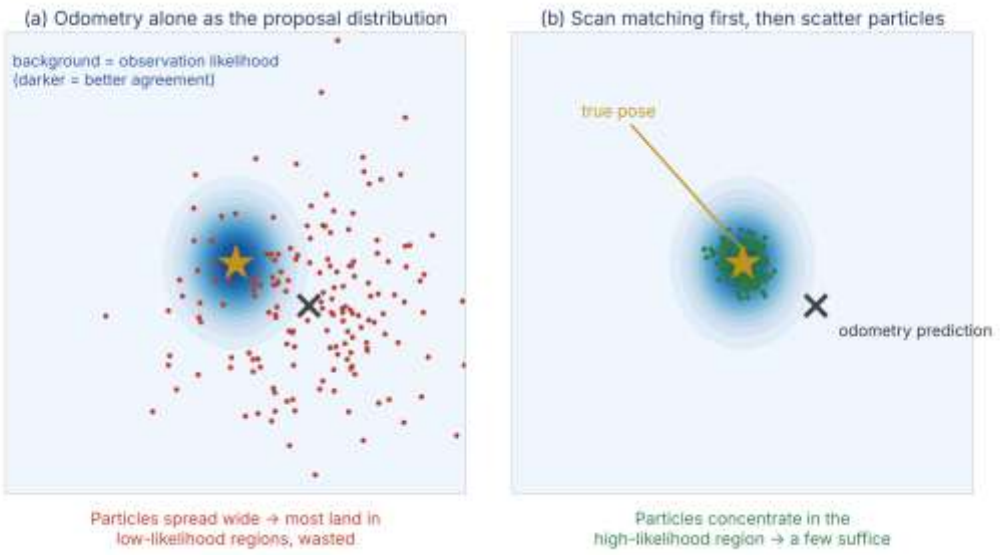
Improvement One: Improving the Proposal Distribution with Scan Matching

The proposal distribution is the probability distribution governing where to scatter the particles at the next step. The plainest approach uses odometry alone: the particles move by odometry and then spread out with random noise added.

The trouble is that odometry noise is often large, so the particles spread widely while the truly correct pose lies in only a small part of that area. Most particles are wasted at obviously wrong positions, so a great many are needed for a few to land by chance in the right place.

Gmapping's improvement is to refine the pose first with the scan matching of Chapter 13 and then scatter the particles over a far smaller area centered on the refined result.

Figure 14-2 Improving the Proposal Distribution



The background of both panels is the same observation likelihood—the darker the color, the better a laser scan taken from that position would agree with the map. The gold star is the true pose and the gray cross the position predicted by odometry, with a clear discrepancy between them.

The left panel is the plain approach: scatter particles about the odometry prediction. Because odometry noise is large, the particles spread over an area far wider than the high-likelihood region, and the great majority land at nearly impossible positions—

they will receive weights near zero and the round of computation is wasted.

The right panel is Gmapping's approach: find the position of highest likelihood first by the scan matching of Section 13.3, then scatter a small handful of particles about it. Nearly every particle lands within the dark region, and every one is a meaningful hypothesis.

The effect in numbers is striking: the particle count required falls from thousands to dozens. This is not merely a saving in computation but the key that let Gmapping run in real time on the embedded computers of its day. Compare this figure with the four steps of the particle filter in Figure 11-4 of Chapter 11 and you will see that what has been improved is step ②, prediction—blind scattering replaced by informed scattering.

The effect of this improvement is striking: it lets Gmapping achieve with dozens of particles what the plain method needs thousands to achieve. You should be able to see that this is the extended application of the "odometry predicts, LiDAR corrects" idea of Section 13.4.

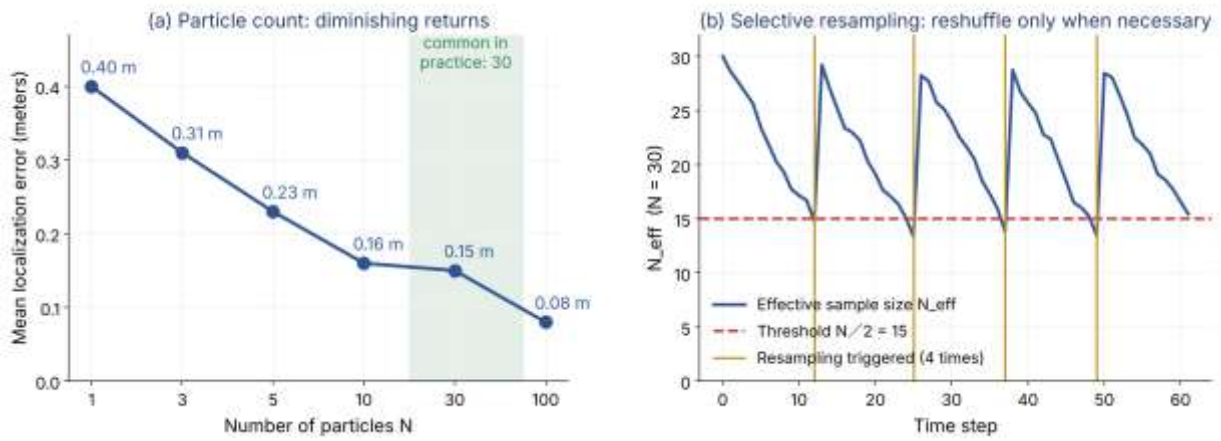
Improvement Two: Selective Resampling

Against the particle depletion problem of Section 14.3, Gmapping adopts selective resampling: rather than resampling at every step, it first assesses the diversity of the particles and resamples only when that diversity has fallen too low.

The measure of diversity is called the effective sample size (ESS). If all particles have similar weights (meaning all are about equally plausible), the ESS is high, diversity is good, and no resampling is needed; if a few particles have very high weights and the rest very low (meaning the contest has been decided), the ESS is low, and only then is resampling performed to concentrate resources.

This strategy strikes a balance between preserving diversity and concentrating on the correct hypothesis, and greatly slows particle depletion. These two improvements—an improved proposal distribution to cut the required particle count, and selective resampling to avoid depletion—are the core contributions of the Gmapping paper and the reason it became a practical system.

Figure 14-3 The Particle-Count Trade-Off and Selective Resampling



The left panel shows actual results: the same trajectory and the same noise settings with only the particle count changed. With 1 particle the mean localization error is about 0.40 meters—which is in fact no filtering at all, just dead reckoning on a single hypothesis; 5 particles bring it to 0.23 meters; 10 to 0.16; 30 to 0.15; and even 100 only to 0.08.

Note that there is almost no improvement from 10 to 30, and that going from 30 to 100 (more than tripling the computation) buys only 0.07 meters. This is diminishing returns—the quantitative version of "thirty particles are both enough and not enough" in Case 14-2. Gmapping's default particle count in practice falls near this sweet spot.

The right panel shows selective resampling at work. The blue curve is the effective sample size N_eff over time: just after a resampling all weights are equal and N_eff

equals the total of 30; as observations accumulate the weight concentrates on a few particles and N_eff falls steadily. Only when it drops below the threshold N / 2 = 15 (the red dashed line) is resampling triggered (the yellow vertical lines), five times over the whole trajectory.

This sawtooth curve makes the particle depletion of Section 14.3 quite clear: resample at every step and diversity is destroyed over and over, so a correct hypothesis may be deleted before it has had a chance to prove itself; never resample and all the weight concentrates on one particle while the rest become ornaments. Selective resampling strikes the balance between these extremes.

14.5 The Procedure for Running Gmapping in ROS

With the theory covered, this section describes the complete procedure for running Gmapping in ROS. It is also a preview of the implementation chapter, Chapter 29.

Prerequisites: Three Things, None of Which May Be Missing

When students fail at their first Gmapping run, the cause is almost always a missing prerequisite:

1. Laser scan data: the LiDAR driver publishing the /scan topic normally (the LaserScan format of Chapter 8).
2. Odometry data: the robot publishing the odom topic and the odom→base_link transform (Section 10.7). This requires wheel encoders and correct wheel-radius and wheelbase parameters.
3. A complete TF tree: the fixed base_link→laser transform above all (the extrinsics of the LiDAR mounting, Chapter 10). Set it wrong and the scan points are

displaced relative to the robot and the map must come out askew—which is the origin of the Chapter 10 maxim that a skewed map means checking TF first.

The Mapping Procedure

4. Start the robot's low-level drivers (motor control, encoder odometry, LiDAR).
5. Confirm the prerequisites: use the tools to check that /scan carries data, that the TF tree is complete and unbroken, and that odometry changes as the robot moves.
6. Launch the Gmapping node and set the key parameters (particle count, map resolution, update distance thresholds, and so on).
7. Open RViz and display the map, the laser scan, TF, and the particle cloud so that mapping can be watched in real time.
8. Teleoperate the robot slowly through the environment, covering every area to be mapped. Go slowly—this is one of the keys to success.
9. Watch the map grow: are the wall lines clear? Is there ghosting? Does the loop close after a circuit?
10. When mapping is complete, save the map with map_saver, which produces a .pgm image file and a .yaml description file.
11. For later navigation, load this map with map_server and localize with AMCL (Chapters 3 and 11).

Key Parameters

Parameter	Meaning	Tuning Advice
Particle count	How many trajectory hypotheses to	30 for a classroom; raise to 50–100 for a large environment or poor odometry

Parameter	Meaning	Tuning Advice
	maintain	
Map resolution	The actual side length each cell represents	0.05 m is common indoors (Section 13.5)
Update distance / angle threshold	How far it must move before an update	Too large misses detail, too small wastes computation
Maximum usable laser range	Measurements beyond this are discarded	Slightly below the LiDAR's nominal range is more robust
Odometry noise parameters	How inaccurate odometry is assumed to be	Raise on a platform that slips badly

Table 14-1 Principal Gmapping Parameters and Tuning Advice

14.6 Common Failures: Wheel Slip, Repetitive Scenes, LiDAR Occlusion

This section may be the most useful in the chapter for implementation. Failure in SLAM is the norm rather than the exception, and recognizing failure modes and treating them appropriately is the core of engineering ability.

Failure One: Wheel Slip and Inaccurate Odometry

Symptom: the map shows clear misalignment or ghosting, especially at turns; the robot believes it has traveled far and the map is stretched.

Cause: wheels slipping on a smooth floor, an abrupt change in friction at the boundary between carpet and tile, or wrong wheel-radius or wheelbase parameters (this last is most often overlooked and produces a systematic scale error that enlarges or shrinks the whole map proportionally).

- Remedy: calibrate wheel radius and wheelbase properly (drive the robot 5 meters straight and turn it 10 times in place, comparing odometry readings with measurements); slow down to reduce slip; and raise Gmapping's odometry noise parameters so the system trusts odometry less and relies more on scan matching.

Failure Two: Repetitive Scenes and Insufficient Features

Symptom: a long corridor drawn now long, now short, with overlapping wall lines; the robot thoroughly lost in a large open hall. This is exactly the corridor difficulty of Section 13.6.

Cause: a lack of distinctive geometric features along the corridor means scan matching cannot determine the amount of longitudinal motion; in open space everything around exceeds the LiDAR's range and there is nothing to match.

- Remedy: ensure odometry is of good quality (it is then the only source of longitudinal information); raise the particle count to maintain more hypotheses; place temporary obstacles such as cardboard boxes along the corridor to provide features (a technique commonly used in teaching); or use a longer-range LiDAR.

Failure Three: LiDAR Occlusion and Invisible Obstacles

Symptom: gaps appear in the map where there should be none; the robot runs into something not drawn on the map.

Cause: glass and mirrors let the laser pass through or reflect it (Chapter 8); black light-absorbing objects reflect weakly; an obstacle is not at the height of the LiDAR's scan plane (the height blind spot of Chapter 8 and the chair-leg problem of Case 8-1); or someone standing in front of the LiDAR creates a moving occlusion.

- Remedy: clear the site while mapping and keep people from walking around the robot; glass partitions may be temporarily papered over or shielded with cardboard boxes during mapping; and recognize that a 2D map reflects only one height slice, so navigation must be supplemented with other sensors.

Failure Four: Moving Too Fast

Symptom: scan matching cannot keep up, and the map is suddenly misaligned or even blows up.

Cause: the robot moves or turns too quickly, the difference between successive scans is too large, and the convergence range of scan matching is exceeded (ICP needs a good initial guess, Section 13.3).

- Remedy: slow down, especially while turning. This is the simplest and most effective measure of all—in teaching experience, most mapping failures are solved by simply driving more slowly.

Failure Mode	Typical Symptom	Principal Remedy
Wheel slip	Map misalignment, ghosting, distorted scale	Calibrate wheel radius and wheelbase, slow down, raise odometry noise
Repetitive scene	Corridor deformed, robot lost	Ensure odometry quality, add particles, add temporary features
LiDAR occlusion	Gaps in the map, hitting obstacles not drawn	Clear the site, deal with glass, recognize the height blind spot
Moving too fast	Map suddenly misaligned or blown up	Slow down, especially while turning

Table 14-2 Common Gmapping Failure Modes and Remedies

14.7 Teaching Practice: Building a Classroom Map with a Small Vehicle

This section describes how to turn Gmapping into a workable class session, with suggestions at two levels of depth for high school and undergraduate courses. Detailed steps appear in this chapter's hands-on activities.

The High School Version: Understand First, Then Do

A high school course should center on observation and experience without going deep into the mathematics:

- Use a simulator first (TurtleBot3 in Gazebo, Chapter 28) so that every student can operate one without being limited by the amount of hardware.
- Focus on three things: how the map grows as the robot moves, how the particle cloud spreads and converges with motion, and how the map goes wrong when driven too fast.
- Demonstrate failure deliberately: have students drive the robot very fast and watch the map blow up before their eyes—the shock of failure builds intuition better than the smoothness of success.
- Connect it to daily life: compare with the map in a home cleaning robot's app (Case 13-1) and see that the two are the same thing.

The Undergraduate Version: Tune the Parameters, Understand the Mechanism

Undergraduates should go further, operating a physical platform and understanding the effect of the parameters:

- Complete the TF configuration and odometry calibration by hand to feel the importance of the prerequisites (this often takes half the debugging time).
- Experiment with the parameters systematically: vary the particle count, map resolution, and odometry noise, and compare map quality quantitatively (by the assessment methods of Section 13.7).
- Map both a classroom and a corridor and check the prediction of Section 13.6—that the corridor is indeed harder.
- After mapping, go on to run AMCL localization and navigation (Chapter 29), completing the full chain of mapping → localization → navigation.

Teaching Notes

From years of teaching experience, three points are recommended to instructors. First, explain the prerequisites and failure modes clearly before letting students start—otherwise the class drowns in debugging. Second, stress going slowly: speed is the single largest variable in a beginner's success rate. Third, be sure to include a step for assessing map quality (Section 13.7), so that students form the habit of not merely building a map but judging whether it is good. These three raise the success rate of a SLAM practical considerably.

You have now been through a first complete SLAM system. Gmapping unties the chicken-and-egg problem elegantly with a particle filter, but it has clear limits: the memory pressure of particle count times map size makes large scenes hard to handle, and a filtering method cannot go back and correct history (Chapter 12). The Cartographer of the next chapter takes another road—using the graph optimization and loop closure detection of Chapter 12 to make large-area mapping and a one-click straightening possible. Comparing the two will give you a deeper understanding of the

trade-off between the filtering and optimization routes.

Formulations and Procedures

1. The Probabilistic Factorization of Rao-Blackwellization

SLAM must estimate the joint distribution over trajectory and map (Chapter 1). The key of the RBPF is to split it into two factors:

$$p(x(1:t), m \mid z(1:t), u(1:t)) = p(m \mid x(1:t), z(1:t)) \cdot p(x(1:t) \mid z(1:t), u(1:t))$$

The second factor on the right is the distribution over trajectories—low-dimensional, sampled with particles; the first is the distribution over maps given a trajectory—which Chapter 13 showed can be computed directly in log-odds with no sampling. This expression is the mathematical statement of the decomposition of Section 14.2 and a distillation of all of Gmapping's ingenuity.

2. The Particle Weight

The weight of the i-th particle is updated as:

$$w^{(i)} \propto p(z_t \mid m^{(i)}, x_t^{(i)})$$

Read: the weight is proportional to the likelihood of observing z_t given that particle's map $m^{(i)}$ and pose $x_t^{(i)}$. The better the scan agrees with the map, the higher the weight. This is the concrete form of the observation model of Chapter 11 within SLAM.

3. Effective Sample Size and Selective Resampling

The measure of particle diversity:

$$N_eff = 1 \;/\; \Sigma_i \, (w^{(i)})^2 \quad \textit{(with the weights normalized)}$$

If all particles have equal weight (diversity at its best), N_eff equals the total count

N; if the weight concentrates on a single particle, N_eff approaches 1. Gmapping's selective resampling rule is:

$$\textit{resample only if } N_{eff} < N / 2$$

This simple test is the key mechanism for avoiding particle depletion in Section 14.4.

4. One Complete Step of Gmapping

Figure 14-4 The Processing of One Gmapping Step

① Obtain the new odometry reading and laser scan

↓

② For each particle: take odometry as the initial guess and run scan matching to refine the pose

↓

③ Sample within a small neighborhood of the refined result for that particle's new pose (the improved proposal distribution)

↓

④ Compute the weight: compare the scan that particle's map predicts with the actual scan

↓

⑤ Each particle updates its own map with its own new pose (log-odds, Chapter 13)

↓

⑥ Compute N_eff; if it is below the threshold, resample (selective resampling)

↓

⑦ Output the map and pose of the highest-weight particle → return to ①

5. Estimating Memory

Gmapping's memory requirement is the particle count times the size of a single map:

$$\textit{total cells} = \textit{particle count} \times (\textit{width} \,/\, \textit{resolution}) \times (\textit{height} \,/\, \textit{resolution})$$

For example, 50 particles, a 20 m × 20 m room, and 0.05 m resolution: 50 × 400 × 400 = 8 million cells. This expression quantifies the limitation noted in Section 14.3—and is the fundamental reason Gmapping scales poorly to large scenes.

14.8 Gmapping Today: Tools Become Obsolete, Ideas Do Not

Let us be honest with students: to build a 2D map in ROS 2 today, most people would not choose Gmapping. It is a package of the ROS 1 era, and the mainstream of the official ecosystem has moved to the slam_toolbox introduced in Section 13.8. Why, then, does this book devote a whole chapter to it?

1. Because the RBPF Has Not Gone Away

The Gmapping package will retire, but the Rao-Blackwellized particle filter behind it is alive in three places.

First, the AMCL of Chapter 16—cleaning robots and service robots localizing in a known map use exactly a particle filter, simply without having to build the map any more. Second, the localization modules of consumer cleaning robots adopt this route extensively, because it performs stably on low computing resources and is simple to implement. Third, it continues to be carried over to new problems: the automatic magnetometer calibration of Section 6.7 is an example—a factorized particle filter estimating pose, magnetic field map, and sensor bias at once, with a skeleton identical to this chapter's.

In other words, what this chapter teaches is not how to use a piece of software but a structure for solving problems: when part of the state can be solved analytically given the rest, factor it out and use particles only on the genuinely hard part. This structure will prove useful again and again in later research.

2. Its Weakness Is Exactly the Subject of the Next Chapter

Several comparative studies of 2D SLAM reach the same conclusion: in corridors, large sites, and scenes requiring loop closure, methods with an optimization back end (the Cartographer family) are clearly better than Gmapping in map accuracy. The reason has been foreshadowed in this chapter—a particle filter is recursive, and once every particle has strayed at some step there is no way back; and resampling itself keeps consuming diversity.

This is not poor implementation but a structural limitation of the filtering route, as the summary of Chapter 11 already said. The Cartographer of Chapter 15 fills that gap with the pose graph optimization of Chapter 12: keep all history and correct it in one pass at a loop closure. Reading the two chapters in sequence gives a felt understanding of why nearly every modern SLAM back end uses optimization, rather than merely a memorized conclusion.

3. The Particle Filter Is Itself Advancing

It is worth noting that research on particle filters has not stood still, and recent improvements run broadly in three directions. First, learning the proposal distribution: a neural network learns from data where to scatter the particles, replacing this chapter's scan matching. Second, parallelization: particles are mutually independent and naturally suited to a GPU, which makes thousands of particles no longer a burden today. Third, hybrids with optimization: particles handle multimodality and uncertainty while optimization handles local refinement—a layered approach visible in LIO-SAM in Chapter 18 and ORB-SLAM3 in Chapter 23.

Three Judgments for Students

1. Do not learn only the tool when learning an algorithm. How slam_toolbox is used will change within a few years, but the structure of factorizing and representing multimodality with particles will not.

2. Filtering and optimization do not displace each other. Section 11.8 said it: filtering for high-rate real-time work, optimization for global consistency; a complete system has both.

3. This chapter's exercise is still worth doing. Even if the final product does not use Gmapping, watching thirty particles converge step by step and N_eff drop below the threshold to trigger a resampling is the fastest route to understanding the whole family of probabilistic SLAM.

Case Studies

Case 14-1 The Original Authors' Classroom: From Paper to YouTube

Gmapping's three authors—Grisetti, Stachniss, and Burgard—are core members of the Freiburg and Bonn probabilistic robotics group introduced in Chapter 2. Interestingly, Professor Stachniss later recorded the complete derivation of particle filter SLAM and published it free on YouTube, where it has become shared teaching material for SLAM learners worldwide.

This means that after finishing this chapter you can go straight to the algorithm's originator explaining it himself—he shows with animations how the particles spread, how they are weighed by observations, and how resampling makes them converge. This combination of reading a textbook for the architecture and watching the original author for the intuition is the best practice of the learning route recommended in Chapter 2.

This case also echoes "the power of open knowledge" in Chapter 2: an algorithm published in a top journal in 2007 is available today to any high school or university student in Taiwan—the original paper, the open-source code, and a lecture by the author, all free. The barrier to learning SLAM has moved from whether one can get into a famous school to whether one has a good map of the subject and sustained persistence—exactly the role this book hopes to play.

Case 14-2 A Classroom of Thirty Particles: Why Gmapping Is Both Enough and Not Enough

A typical observation in university teaching experiments: mapping an ordinary classroom with 30 particles gives a quite beautiful result—straight wall lines, a closed loop, clear detail. But take the same settings to a whole floor (tens of meters across, several rooms and a long corridor) and map quality falls noticeably, or fails outright.

The reason is twofold. First, memory and computation: as the scene grows, the cell count of each map explodes, and multiplied by the particle count the burden is heavy (formulation 5). Second, and more fundamental: a larger scene means a longer trajectory and more accumulated drift, requiring more particles to cover the possible hypotheses; but with more particles the computation cannot keep up—a dilemma.

More crucially: when the robot completes a large circuit back to its starting point, even if the loop closure is detected, a particle filter cannot go back and correct the whole historical trajectory as graph optimization can—every particle's map was long since drawn along a wrong trajectory and cannot be redone (the central argument of Chapter 12). This is exactly the problem Cartographer in Chapter 15 sets out to solve. Performing this "classroom succeeds, whole floor fails" comparison with your own hands gives a deep sense of the difference between the two chapters.

Case 14-3 Particles Inside the Cleaning Robot: The Theory Is in Your Home

A case in Chapter 11 noted that many cleaning robots have a debug mode in which the particle cloud can be seen converging. This chapter takes it further: a cleaning robot's first mapping and its later cleanings in fact correspond to two different techniques.

Entering an unfamiliar environment for the first time, the robot must localize and map at once—the SLAM of this chapter. Once the map is built, every later cleaning need only localize within a known map, which is the Monte Carlo localization (AMCL) of Chapters 3 and 11. Both share the particle filter skeleton and differ in whether the map is known: SLAM's particles each carry a growing map, while AMCL's particles share one fixed map.

Understanding this distinction matters, because it corresponds to two different tools and procedures in ROS (Section 14.5): run Gmapping during mapping and save the map; load the map with map_server during navigation and run AMCL to localize. The exercises of Chapter 29 walk through this chain in full. Next time you watch the cleaning robot at home explore gingerly on its first run and set to work confidently on its second, you will know it has switched algorithms.

Hands-On Activities

Exercise 14-1 Building a Classroom Map with Gmapping

Purpose: to run a complete particle filter SLAM mapping session, observe the evolution of the particle cloud, and understand Gmapping's mechanism and limits through parameter experiments and reproduced failures. The exercise offers both a simulation

route and a hardware route.

Route A: Mapping in Simulation (the whole class, no hardware needed)

1. Launch the TurtleBot3 Gazebo simulation environment and the SLAM node (Chapter 28 has the complete installation steps).
2. Open RViz and display at once: the occupancy grid map, the laser scan, the TF axes, and the particle cloud (if the tool supports it).
3. Teleoperate the robot slowly around the simulated room with the keyboard, watching the map grow cell by cell and the unknown gray region fill in as you go.
4. Observe the particle cloud: how does it spread while the robot drives straight? How does it converge at a turn or on seeing a distinct feature? Compare with the theory of Chapter 11 and Section 14.3.
5. Speed experiment: map once very slowly, then map again at top speed, compare the quality of the two maps, and reproduce the "moving too fast" failure of Section 14.6.
6. Save the map and assess its quality by the methods of Section 13.7: are the walls straight? Does the loop close? Measure the room dimensions on the map against the simulation settings.

Route B: Mapping with a Physical Vehicle (for classes with hardware)

7. Pre-checks: confirm that /scan carries data, that the TF tree is complete (base_link→laser above all), and that odometry changes with motion. This step may not be skipped.
8. Odometry calibration: drive the vehicle 3 meters straight and compare the odometry reading with a tape measurement; turn it 10 times in place and

compare the angle reading. If the error is large, correct the wheel-radius and wheelbase parameters.

9. Clear the site: ask students to withdraw to the edges of the room, remove objects that might occlude the LiDAR, and temporarily paper over any glass partitions.
10. Teleoperate the vehicle slowly once around the classroom (including all four corners), return to the starting point, and observe whether the loop closes.
11. Save the map files (.pgm and .yaml), measure the classroom dimensions on the map against tape measurements, and compute the percentage error.

Part Two: Parameter Experiments (Undergraduate)

12. Particle count experiment: map the same site with 5, 30, and 100 particles and compare map quality against computational burden (observe CPU usage and update rate).
13. Resolution experiment: map at resolutions of 0.02, 0.05, and 0.10 m and compare detail against memory use, verifying the estimate of formulation 5.
14. The corridor challenge: drive the robot into a long corridor and map it, checking the predictions of Sections 13.6 and 14.6 and observing whether distortion appears. Try placing cardboard boxes along the corridor to provide features, map again, and compare the improvement.
15. Slip experiment: lay a smooth board on the floor or lift one wheel briefly to induce slip, and observe how the map goes wrong.

Discussion Questions

1. How far do the maps built at high and low speed differ? From the standpoint of Section 14.6 and Section 13.3, explain why driving slowly is so effective.

2. Going from 5 particles to 100, how much did map quality improve? How much did the computational burden increase? What do you think is the most suitable particle count for your site?
3. What went wrong mapping the corridor? Did the cardboard boxes improve it? Which claim of Section 14.6 does this confirm?
4. If the robot completes a large circuit back to its starting point and the map does not close, can Gmapping correct the whole map after the fact? Why not? (Hint: Chapter 12 and Case 14-2.)

Extended Challenge (Undergraduate)

First: implement a minimal RBPF simulation in Python—define a virtual room in code, create 20 particles (each with a pose and a small map), implement the loop of move with noise → weigh → selectively resample → each updates its own map, draw every particle's map side by side, and watch the correct hypothesis win. Second: after mapping, go on to launch map_server and AMCL so the robot localizes in the map it built and performs navigation (Chapter 29), completing the full chain of mapping → localization → navigation.

Further Reading

1. G. Grisetti, C. Stachniss, and W. Burgard, "Improved Techniques for Grid Mapping with Rao-Blackwellized Particle Filters," IEEE Transactions on Robotics, 2007. (The original Gmapping paper and the principal basis of this chapter.)
2. M. Montemerlo et al., "FastSLAM: A Factored Solution to the Simultaneous Localization and Mapping Problem," AAAI, 2002. (The pioneering work on RBPF-SLAM, a useful counterpart to Gmapping.)

3. S. Thrun, W. Burgard, and D. Fox, Probabilistic Robotics, MIT Press, 2005. (Chapter 13 gives the complete theory of FastSLAM and particle filter SLAM.)
4. Cyrill Stachniss, the Particle Filter and FastSLAM course videos (YouTube). (Explained by the original author; best read alongside this chapter.)
5. The OpenSLAM Gmapping project and the documentation of the ROS slam_gmapping package. (Original source code and parameter descriptions.)
6. The slam_toolbox open-source project (GitHub). (Modern 2D SLAM based on graph optimization in ROS 2; useful for comparative experiments against Gmapping.)
7. The SLAM chapters of the ROBOTIS TurtleBot3 e-Manual. (Complete operating instructions for mapping in simulation and on hardware.)

From Particles to Graphs: Where This Chapter Stands

This chapter has shown SLAM working properly for the first time: pose and map estimated together, and estimated rather well. It is the first complete system of Part IV.

But the curve in the left panel of Figure 14-3 states its ceiling honestly. Even raising the particle count to a hundred brings the mean error only to 0.08 meters, and the computation is already more than triple that of thirty. The more fundamental problem is that every correction a particle filter makes happens in the present—it has no mechanism whatever for going back and rewriting the trajectory it has traveled.

Imagine the robot completing a circuit of a whole building back to its starting point and finding that what it sees does not match the starting point recorded on the map. All Gmapping can do is correct slowly from that moment on; the error of the whole circuit, already written into every particle's map, can never be taken back out. This is the

drift symptom of Section 13.7, and its remedy is loop closure.

The Cartographer of Chapter 15 solves this with three things: accumulating scans into submaps to reduce the size of the pose graph, searching the whole map efficiently for loop closure candidates by branch and bound, and then correcting the positions of all submaps in one pass by the pose graph optimization of Chapter 12. In the same classroom with the same vehicle, Chapter 15 will let you see for yourself how the two maps differ—and looking back at this chapter then, both the merits and the limits of Gmapping will have become very concrete.

Summary

1. Gmapping was proposed by Grisetti, Stachniss, and Burgard in 2007 and is one of the best-known 2D laser SLAM systems; its inputs are laser scans, odometry, and TF, and its outputs an occupancy grid map and the map→odom transform.
2. Rao-Blackwellization splits SLAM in two: the trajectory (low-dimensional, sampled with particles) and the map (solvable analytically given the trajectory); each particle is a parallel world carrying one trajectory and one map, competing on observation scores and untying the chicken-and-egg problem elegantly.
3. The particle count trades accuracy against computation (about 30 for a classroom); the weight represents how well a hypothesis agrees with the observation; resampling lets good hypotheses survive, but excessive resampling causes particle depletion, letting a wrong hypothesis monopolize the set irrecoverably.
4. Gmapping's two key improvements: improving the proposal distribution with scan matching (scattering particles at plausible positions and cutting the

required count from thousands to dozens), and selective resampling (resampling only when the effective sample size falls below a threshold, to avoid depletion).

5. The three prerequisites of a ROS implementation, none of which may be missing: /scan laser data, odometry with the odom→base_link transform, and a complete and correct TF tree (the base_link→laser extrinsics above all); after mapping, save with map_saver as .pgm and .yaml.
6. The four major failure modes: wheel slip (calibrate wheel radius and wheelbase, raise odometry noise), repetitive scenes (corridors—rely on odometry and add features), LiDAR occlusion (glass, black objects, the height blind spot), and moving too fast (slowing down is the most effective single measure).
7. Gmapping's fundamental limitation is that memory grows as particle count times map size, which scales poorly to large scenes; and as a filtering method it cannot go back and correct the historical trajectory and map—exactly the problem the optimization route of the next chapter sets out to solve.

This chapter completed a first full SLAM system. Gmapping unties the chicken-and-egg problem elegantly with a particle filter, but it also shows us the ceiling of the filtering route plainly: large scenes are too much for it, and history cannot be corrected. The next chapter brings in Cartographer—developed by Google, it organizes space into submaps, performs loop closure correction with the graph optimization of Chapter 12, maps large areas in real time, and straightens a whole skewed map at one stroke the instant a loop closure is detected. That striking picture from Chapter 12 is about to reappear in a real system.

A Taiwan-Authored University Textbook

@prof

Simultaneous Localization and Mapping

Simultaneous Localization and Mapping, SLAM

Chapter 15

Cartographer: Google's 2D / 3D SLAM

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

At this point in Part IV, each of the two main lines of theory has at last grown into a complete system. The Bayesian filtering of Chapter 11 grew into Gmapping in Chapter 14; the graph optimization of Chapter 12 grows into Cartographer in this chapter. The two chapters are not parallel substitutes but two answers to the same problem within two mathematical frameworks, which is why reading them together yields twice what reading them separately would.

Looking back, this chapter draws on nearly the whole stock of the first three parts: the LiDAR specifications of Chapter 8 determine the quality of the material for scan matching, the IMU of Chapter 7 is all but a necessity in 3D mode, the SE(2) pose composition of Chapter 10 is the underlying language of stitching submaps, the probabilistic grid idea of Chapter 11 was made concrete in Chapter 13 as the internal representation of a submap, and the sparse pose adjustment of Chapter 12 is the engine of this chapter's global SLAM itself. Looking forward, the .pbstream maps and loop closure correction produced here will be drawn upon again and again in the navigation of Part V, the multi-robot work of Part VI, and the field applications of Part VII.

Figure 15-0 The Connection Map of Chapter 15

The stock of theory from the first three parts

Chapter 7 IMU Chapter 8 LiDAR Chapter 10 SE(2) pose

| | |

└──────┼──────┘

▼

Ch. 11 Probabilistic filtering ── Ch. 13 Occupancy grid ── Ch. 12 Graph optimization

| | |

▼ ▼ ▼

Chapter 14 Gmapping [Chapter 15 Cartographer]

(the filtering route) (the optimization route, this chapter)

|

▼

Part V Navigation Part VI Multi-robot collaboration Part VII Field applications

Learning Objectives

The Gmapping of the previous chapter untied the chicken-and-egg problem with a particle filter but also exposed the ceiling of the filtering route: large scenes are too much for it, and history cannot be corrected. This chapter introduces Cartographer, developed by Google—it organizes space into submaps, performs loop closure correction with the graph optimization of Chapter 12, maps large areas in real time, and straightens a whole skewed map at one stroke the instant a loop closure is detected. After completing this chapter, you should be able to:

1. Describe Cartographer's two-subsystem architecture and the division of labor between local SLAM and global SLAM.
2. Explain the concept of a submap and why scans are grouped into submaps.

3. Explain the purpose of and difference between local and global scan matching.
4. Explain how branch and bound makes real-time loop closure detection possible.
5. Describe the procedure and key settings for running 2D Cartographer mapping in ROS.
6. Describe 3D Cartographer and its application to backpack spatial modeling.
7. Compare Cartographer with Gmapping in architecture, strengths and weaknesses, and suitable settings.

Core Concepts

Chinese Term	English Term	Explanation
子地圖	Submap	A small probability grid map made of a few dozen scans; Cartographer's basic unit.
局部 SLAM	Local SLAM	The front end: inserts new scans into the current submap; locally consistent but drifting.
全域 SLAM	Global SLAM	The back end: seeks loop closure constraints in the background and optimizes the whole pose graph.
機率格	Probability Grid	The internal representation of a submap, each cell recording an occupancy probability.
掃描匹配	Scan Matching	The process of aligning a new scan to the current or an existing submap.
閉環約束	Loop Closure Constraint	An edge connecting revisited places, used to eliminate accumulated error.

Chinese Term	English Term	Explanation
分支定界	Branch and Bound	An algorithm that searches efficiently for the best match and rejects impossible solutions.
多解析度預算格	Precomputed Grids	Maps precomputed at several resolutions to accelerate the branch-and-bound search.
稀疏位姿調整	Sparse Pose Adjustment	The optimization performed on the graph formed by submap and scan poses.
非線性最小平方	Nonlinear Least Squares	The form of optimization Cartographer solves with Ceres.
背包式建模	Backpack Mapping	Carrying a sensor suite on foot to scan the space of a building.
穩健核函數	Robust Kernel	An optimization technique that reduces the influence of a false loop closure.

15.1 The Architecture of Cartographer

Cartographer was developed by Google and open-sourced in 2016, and the title of its core paper names its headline feature directly: "Real-Time Loop Closure in 2D LIDAR SLAM." That name itself declares the fundamental difference from Gmapping—loop closure detection and graph optimization are built into the core of the system, and they run in real time.

Two Subsystems: Local and Global

The most important thing in understanding Cartographer is to grasp its two-subsystem architecture. It may be regarded as two independent but related subsystems: local SLAM (also called the front end or the local trajectory builder) is responsible for building a sequence of submaps, each locally consistent, and we accept that local SLAM will drift over time; global SLAM runs in a separate thread and looks for loop closure constraints among the submaps.

- Local SLAM (the front end): matches each new scan and inserts it into the current submap. It aims at real-time operation and local accuracy, in the full knowledge that drift will accumulate over time—a deliberate design trade-off.
- Global SLAM (the back end): in a background thread, matches scans against nearby submaps to establish loop closure constraints, and continuously optimizes the constraint graph formed by submap and scan poses.

The key is that both run at once. The foreground maps quickly so the operator sees results immediately, while the background quietly seeks loop closures and restores global consistency. This is why it is real time, and the greatest architectural difference from Gmapping.

Figure 15-1 Cartographer's Two-Subsystem Architecture

[Local SLAM (the front end)—running in the foreground]

New scan → scan matching to the current submap → insert into the submap → seal the submap when complete

Character: real-time, locally consistent, drifting

[Global SLAM (the back end)—a background thread]

Scan ↔ matched against nearby existing submaps → a good match establishes a loop closure constraint

↓

Submap poses and scan poses form a pose graph (Chapter 12)

↓

Every few seconds, solve the nonlinear least-squares problem with Ceres → the whole map is straightened

Both run at once: the foreground gives a real-time preview, the background guarantees global consistency

The Connection to Chapter 12

You should recognize it at once: what global SLAM does is exactly the graph optimization of Chapter 12. It combines submaps formed by local scan matching (probability grids refined by a nonlinear least-squares optimizer based on Ceres) with

a sparse pose graph carrying branch-and-bound loop closure detection, achieving accurate real-time 2D SLAM on ordinary hardware. Chapter 12's mention that Cartographer uses the Ceres library is confirmed here. Theory and system formally meet in this chapter.

15.2 Submaps

The submap is Cartographer's most distinctive design. Understand why it exists and you understand why Cartographer can handle large scenes.

What a Submap Is

A submap is a small probability grid map (the occupancy grid of Chapter 13) accumulated from a few dozen consecutive laser scans. As the robot moves, new scans are continuously inserted into the current submap; once the configured number of scans has accumulated, that submap is finished and sealed, and the system begins building the next. The environment is thus cut into a series of partly overlapping small maps.

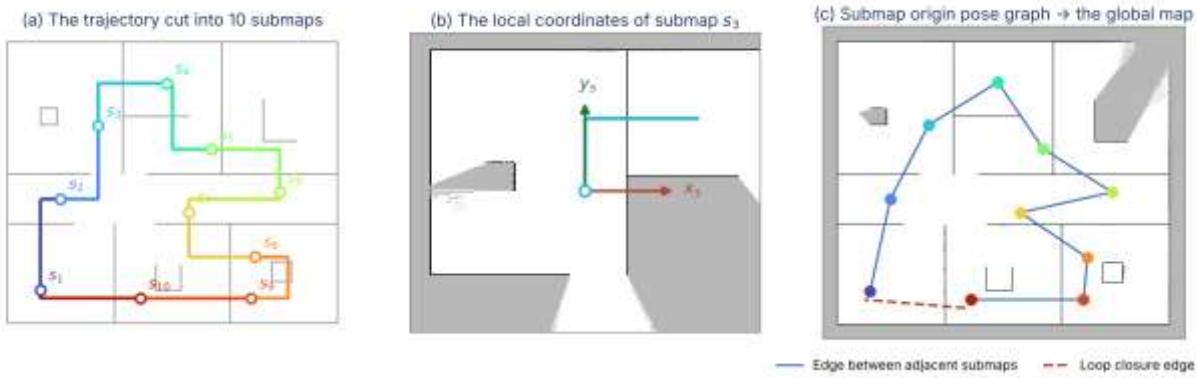
Why Submaps: Three Reasons

1. Bounding the accumulation of error: because a scan is matched only against a submap containing a few recent scans, this approach accumulates error slowly; over merely a few dozen consecutive scans the accumulated error is very small. In other words, within a submap's brief lifetime the drift is negligible—each submap is internally clean.
2. Providing a unit for optimization: in global optimization there is no need to adjust every scan, only the poses of the submaps. One submap represents

dozens of scans, which reduces the node count of the pose graph sharply and lets the optimization finish in real time. This is the key to efficiency.

3. Serving as the object of loop closure comparison: in loop closure detection, the current scan is compared against past submaps. A submap carries more information than a single scan and is more stable, so the comparison is more reliable.

Figure 15-2 Three Aspects of the Submap: Segmentation, Local Coordinates, Stitching



This figure breaks the abstract word "submap" into three aspects the eye can confirm. The left panel shows one trajectory cut into ten segments, each a submap s_1 through s_{10}; the open circles mark each submap's origin pose, the position and heading of the robot when that stretch began. Note that the division is not by distance traveled but by how many scans have accumulated—Cartographer seals a submap by scan count precisely to keep the internal error of each submap at a negligible magnitude.

The middle panel takes the third submap on its own and places it in its own local coordinate frame. The red and green arrows are that submap's x and y axes, with the origin always at the start of that stretch. Inside the submap there is no knowledge whatever of where it is in the world; it remembers only where this wall and that corner are relative to its own origin. This deliberate amnesia is the key to Cartographer's ability

to cut a large scene small: let the local be local and the global be global.

The right panel is the stitched result. Blue lines join the origins of adjacent submaps into a pose chain; the red dashed line is the loop closure edge, pulling the last submap back to the first. The variables actually optimized are only the ten colored dots in the figure—the poses of ten submap origins, thirty unknowns in all. By comparison this trajectory itself has nearly sixty time steps; optimizing the pose of every scan would double the unknowns, and the computation of the optimization grows roughly with their number. This is the concrete meaning of "providing a unit for optimization" in Section 15.2.

Take the three panels together and the submap's role is clear: it is at once an isolation chamber for error (local error does not leak out), a Lego brick for optimization (one brick representing dozens of scans), and the object of loop closure comparison (carrying enough information to be more stable than a single scan). One design solves three problems at once, and that is the engineering taste most worth learning from Cartographer.

How Large Should a Submap Be?

Too small: the number of submaps explodes, the pose graph gains nodes, and global optimization slows; and each submap contains too few geometric features, so loop closure comparison misjudges easily.

Too large: the scan matching error accumulated within a submap becomes significant, the premise that local error is negligible is broken, and wall lines remain askew even after loop closure straightening.

A practical starting point: indoor 2D sites commonly seal a submap every 70 to

100 scans (num_range_data in Table 15-1); reduce it where features are sparse, as in corridors, and raise it in cluttered rooms.

How to judge: look at the overlap between submaps in RViz. If the wall lines of adjacent submaps coincide, the size is right.

The Contrast with Gmapping

The philosophical difference between the two systems is clear here. Gmapping has every particle maintain one complete map (Chapter 14), with memory growing as particle count times map size; Cartographer maintains a single map stitched from many submaps and restores global consistency by adjusting submap poses.

The consequence is direct: Cartographer's memory requirement is proportional to the size of the scene (not multiplied by a particle count), so it can handle a whole building; and because it keeps the submaps and the pose graph, it can go back and correct—exactly the core advantage of graph optimization over filtering noted in Chapter 12.

15.3 Scan Matching

There are two kinds of scan matching in Cartographer with quite different purposes. Keeping them apart is the key to understanding how the system works.

1. Local Scan Matching: Insertion into the Current Submap

This is the front end's everyday work. For each new scan the system must decide where it should be pasted into the current submap. It takes an initial guess from odometry or the IMU (the idea of Section 13.4) and refines it by optimization so that the scan points agree as closely as possible with the submap's probability grid.

Cartographer formulates this match as a nonlinear least-squares problem and solves it with Ceres (Chapter 12). This differs from traditional ICP (Section 13.3)—rather than finding point-to-point correspondences, it directly maximizes the extent to which scan points fall into cells of high occupancy probability, which is less sensitive to noise.

2. Global Scan Matching: Finding Loop Closures

This is the back end's work and far harder. The question is: does the current scan agree with some submap built long ago? If it does, a loop closure has been found.

The difficulty lies in the search space. A loop closure candidate may be at any position and any angle, and because drift has already accumulated the initial guess may be badly off—the search cannot be confined to a small neighborhood. Brute-force search over every possible position and angle would be far too expensive to run in real time.

This is the problem Cartographer's most ingenious technical contribution addresses, and the subject of the next section.

Practical Considerations in Bounding the Search

Even with an efficient algorithm, the system must control the amount of computation. To limit the number of constraints (and the computation), Cartographer considers only a subsampled set of all nearby nodes when building constraints, governed by a sampling ratio constant; sample too little and constraints may be missed and loop closure fail, sample too much and global SLAM slows and real-time loop closure becomes impossible. This is an important trade-off in tuning.

15.4 Loop Closure Detection and Global Optimization

This section is the technical core of the chapter: how Cartographer achieves real-

time loop closure—exactly the achievement its paper's title claims.

Branch and Bound: Pruning Impossible Branches

Faced with a vast search space, Cartographer uses branch and bound. It operates on grid maps at several resolutions with a branch-and-bound mechanism that efficiently rejects incorrect matches.

The central idea of branch and bound can be understood by an everyday analogy: suppose you must find a person in a whole building. The foolish method is to search every room; the clever one is to glance quickly over each floor first (low resolution), and if some floor obviously cannot hold anyone (the whole floor is under renovation, say), skip it entirely without searching room by room—that is pruning.

Cartographer does precisely this: it downsamples the existing submaps to several resolutions (a quarter and a half size, say) as inputs to scan matching, begins the match at low resolution with a coarse computation, and uses the best preliminary score to skip a great many impossible translation-rotation combinations at higher resolution, repeating until full resolution is reached. The system precomputes multi-resolution grids of sizes 1, 4, 16, and 64 to support this search.

Why This Achieves Real-Time Operation

The key is the notion of an upper bound. The score computed on a low-resolution map is an upper bound (never an underestimate) on all possible match scores for that region at high resolution. So if a region's score at low resolution is already below the best answer found so far, it can never do better at high resolution—the whole region can be skipped safely without being expanded.

This reduces the search from all possible combinations to a few promising

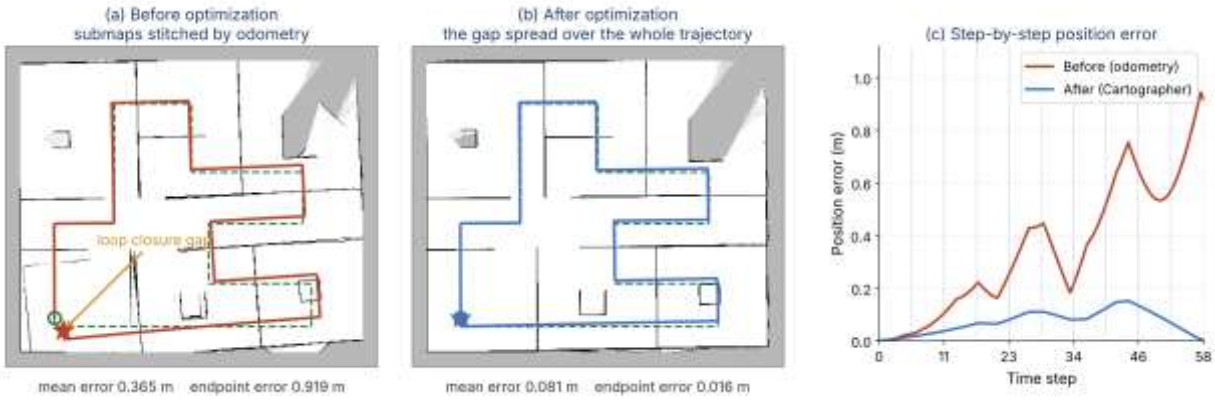
branches, and real-time loop closure becomes possible. The mechanism is explained in detail in the Cartographer paper, and its dedicated fast correlative scan matcher was designed precisely to make scan matching for real-time loop closure possible.

Global Optimization: Spreading the Contradiction

Once loop closure constraints are found, we enter the graph optimization of Chapter 12. Loop closure optimization, like scan matching, is formulated as a nonlinear least-squares problem, which lets the system add residuals easily to incorporate further data; every few seconds the system computes a solution with Ceres.

What this optimization adjusts are the submap poses and the scan poses, and it is called sparse pose adjustment. When the loop closure constraint and the odometry constraints contradict each other, the optimization spreads the contradiction evenly over the whole trajectory—the picture from Chapter 12 of a whole skewed map snapping straight is played out here for real.

Figure 15-3 Before and After Loop Closure: How One Constraint Straightens a Whole Map



This is the figure in the chapter most worth pausing over, because it turns "the most striking three seconds in class" of Case 15-1 into quantifiable numbers. Panel (a) is before optimization: the submaps are stitched together purely by odometry's dead

reckoning. The whole map is visibly skewed, the outer walls rotated bodily along a diagonal, and the end of the robot's circuit (the star) has not returned to the starting point (the circle), the two differing by 0.92 meters. The gold arrow points to that gap—not the sensor's fault but the result of dozens of tiny angular errors accumulating along the way.

Panel (b) is after one loop closure edge has been added and the pose graph solved afresh. Note the nature of the change: the end point is not forcibly dragged back to the start; rather, that 0.92 meters of contradiction is spread back over the whole trajectory according to the confidence of each constraint. The result is a map turned square with wall lines converging into single lines, the endpoint error falling to 0.016 meters and the mean error from 0.365 to 0.081—a reduction of nearly eighty percent. This is what "the covariance decides who yields" from Section 12.3 looks like on real data.

Panel (c) plots the step-by-step error of both trajectories together and is the most instructive panel of all. The red curve (before optimization) shows the classic shape of drift: nearly zero at first, rising gently in the middle, and shooting up at the end—error grows not linearly but as an angular deviation is amplified by the remaining distance. The blue curve (after optimization) is pressed into a gentle arch: the maximum appears in the middle of the trajectory (about 0.15 meters) while both ends, anchored by the start and the loop closure edge, approach zero.

This arch is worth remembering, for it is the fingerprint of pose graph optimization. When a trajectory has strong constraints at both ends and only odometry strung between, the error distribution must be low at the ends and high in the middle. It also tells us something practical: if the site permits two or more loops, there will be two or more loop closure edges and the arch will be pressed lower still. That is why the

mapping procedure of Section 15.5 insists on completing the whole loop, and it foreshadows the comparative experiment of the next section.

Loop Closure Straightening and the Map Suddenly Jumping

Why it jumps: global optimization runs every few seconds, and once a loop closure constraint is found the poses of the whole set of submaps are updated at once, so on screen the whole map rearranges instantly—the three seconds of Case 15-1.

Why navigation must be careful: the map and the map→odom transform change at the same moment, and a path being executed may become invalid instantly. The navigation stack of Part V must tolerate such a discontinuity, which is why mapping and navigation are commonly separated into two phases in practice.

How to confirm straightening succeeded: look at the seam of the loop. Before straightening the wall will ghost or be offset (the symptoms of Section 13.7); after, it should converge into a single line.

What to do if it does not straighten: usually the loop closure score threshold is too high or the sampling ratio too low (Table 15-1), so loosen the threshold and try again; but too loose a threshold invites false loop closures, so tune it together with the robust kernel of Section 15.4.

Robustification: Preventing the Catastrophe of a False Loop Closure

Loop closure detection can err (two rooms that look alike, say). One wrong loop closure constraint will drag the map forcibly to a wrong position with catastrophic consequences (mentioned in the Chapter 12 hands-on activity). Cartographer

therefore uses a robust kernel in the optimization to downweight outlying constraints, so that a few bad matches cannot ruin the whole map. This echoes the Chapter 12 extended challenge of adding a false loop closure edge—now you know how a practical system guards against it.

15.5 Implementing 2D Cartographer

This section covers the practical points of running 2D Cartographer mapping in ROS. Read alongside the Gmapping procedure of Section 14.5, the similarities and differences become clear.

Prerequisites

The same three requirements as Gmapping still apply: laser scans (/scan), odometry, and a complete TF tree (Chapter 10). But Cartographer differs in two respects:

- The IMU matters more: Cartographer can make effective use of the attitude information an IMU provides (Chapter 7), and in 3D mode it is all but required. In 2D mode, having an IMU improves mapping stability noticeably.
- Odometry is not strictly necessary: because its local scan matching is stronger, Cartographer can work without odometry in feature-rich environments. But where features are sparse, as in corridors (Section 13.6), odometry remains critical.

Configuration Files: Lua

A distinctive feature of Cartographer (and a barrier for beginners) is that it uses configuration files in the Lua language, with far more parameters than Gmapping. Most local SLAM options are found in the trajectory_builder_2d.lua and

trajectory_builder_3d.lua configuration files for 2D and 3D respectively.

The key parameter groups include:

Parameter Group	What It Controls	Direction of Tuning
Submap scan count	How many scans accumulate in each submap	Too few makes the submap unstable, too many lets drift into the submap
Resolution	The cell side length of the probability grid	0.05 m is common indoors (Section 13.5)
Local matching weights	The relative importance of each residual in scan matching	Raise the odometry weight where odometry is dependable
Loop closure sampling ratio	How many nodes are considered in building constraints	Too low misses loop closures, too high slows real-time operation (Section 15.3)
Loop closure score threshold	How high a match score is accepted as a loop closure	Too low misjudges easily, too high detects no loop closures
Optimization period	How often global optimization runs	Usually once every few seconds

Table 15-1 Principal Parameter Groups of 2D Cartographer

The Mapping Procedure

4. Confirm that sensor data and the TF tree are normal (as in Section 14.5).
5. Prepare the Lua configuration file, adjusting the sensor configuration to the platform (IMU or not, odometry or not, number of LiDARs).
6. Launch the Cartographer node and RViz, displaying the submaps, the trajectory, and the laser scan.

7. Teleoperate the robot slowly to map. Pay particular attention on completing the loop back to the starting point to whether a loop closure is triggered—the moment in this chapter most worth watching.
8. When mapping is complete, finish the trajectory and run the final optimization, then export the map file (in .pbstream format, convertible to the ROS standard .pgm and .yaml).

What to Watch: Submaps and Loop Closure

In RViz, Cartographer displays the individual submaps distinguishably, so you can clearly see that the map is stitched together from submaps. The finest moment is the instant a loop closure triggers: a trajectory and map that were offset suddenly jump into the correct position. Be sure to allow class time for students to witness this moment—it is the most powerful concretization of the abstract theory of Chapter 12.

15.6 3D Cartographer and Backpack Spatial Modeling

Cartographer does not do only 2D. It also supports 3D SLAM, and one of Google's original motivations for developing it was precisely the three-dimensional surveying of indoor spaces.

How 3D Mode Differs

- Sensors: a 3D LiDAR (multi-beam, Chapter 8) is required. Google's system originally carried two Hokuyo laser scanners and an inertial measurement unit, and later versions used two Velodyne PUCK scanners.
- The necessity of an IMU: in 3D mode the estimation of attitude (roll and pitch, Chapter 7) is crucial, and an IMU is all but required.

- Submaps become 3D: the probability grid goes from planar to volumetric voxels, and memory requirements rise sharply.
- Computational burden: 3D scan matching and optimization cost far more than in 2D, and demand more of the hardware.

Backpack Spatial Modeling

Case 8-3 of Chapter 8 mentioned backpack mobile scanning, and Cartographer is the representative of such systems. An operator carries a backpack fitted with LiDAR and an IMU and builds a three-dimensional model of a whole building in real time while walking.

The application matters because it solves the pain point of traditional surveying: a conventional static laser scanner must be set up at each station, scan, and be carried to the next—slow and laborious; a backpack system scans wherever it walks, raising efficiency tens of times over.

- Applications: building information model (BIM) creation and verification, digital preservation of historic sites and cultural assets, plant facility management, producing indoor navigation maps, and post-disaster structural assessment.
- What it means for university sites: this is a project direction of great potential—scan campus buildings, laboratories, or plants with 3D SLAM to produce a digital twin for teaching and research, putting the 3D point cloud techniques of Part V (Chapters 17 and 18) to practical use.

The Extension to Continuous-Time SLAM

It is worth mentioning that the academic community continues to improve on Cartographer's foundation. Some research groups have pointed out, for instance, that treating scans as a continuous-time trajectory rather than snapshots at discrete

instants further improves 3D mapping quality—work built precisely on a deep understanding of Cartographer's architecture. For students intending to do research, understanding this chapter is the ticket into the field.

15.7 Comparing Cartographer with Gmapping

The two protagonists of this part have now both been introduced. This section makes a systematic comparison, and it is the most important table in Part IV.

Item of Comparison	Gmapping (Chapter 14)	Cartographer (this chapter)
Theoretical basis	Particle filter (Chapter 11)	Graph optimization (Chapter 12)
Map organization	One complete map per particle	A series of submaps stitched together
Loop closure handling	No dedicated mechanism; hard to correct afterward	A core feature; can straighten the whole map in real time
Memory	Particle count × map size	Proportional to the size of the scene
Large scenes	Limited	Its strength; can map a whole building
3D support	None (2D only)	Supports 2D and 3D
Use of an IMU	Not used	Integrates effectively; all but required in 3D
Parameter complexity	Fewer; easy to start with	More (Lua configuration files); a higher barrier
Teaching value	The best example for understanding particle filters	The best example for understanding graph optimization and loop closure

Table 15-2 A System Comparison of Cartographer and Gmapping

Figure 15-4 Same Site, Same Route: The Two Systems Head to Head

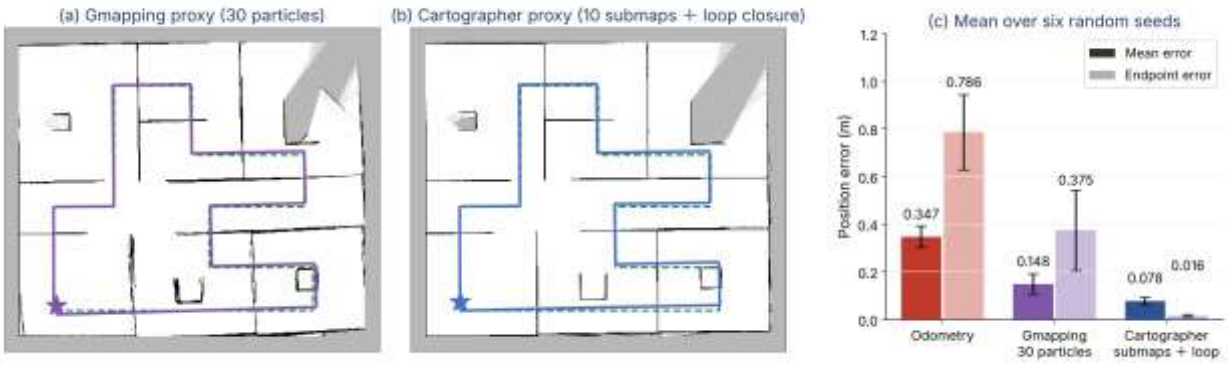


Table 15-2 lists the architectural differences between the two systems; this figure converts those differences into numbers. The experimental conditions are identical: the same 12 × 9 meter L-shaped indoor site, the same route once around and back to the start, and the same odometry noise parameters. Panel (a) is a Gmapping proxy from Chapter 14 (30 particles), panel (b) a Cartographer proxy from this chapter (10 submaps plus one loop closure edge), and panel (c) the statistics over six different random seeds.

Look at the maps first. Both are far better than the odometry-only result, but they differ at the seam: Gmapping's map still shows a slight misalignment where the loop joins, and the outer walls carry a slight overall tilt; Cartographer's map has straight walls and a cleanly closed loop. The reason is in the numbers of panel (c)—Gmapping's endpoint error averages 0.375 meters with a standard deviation of 0.168, while Cartographer's is 0.016 meters with a standard deviation of only 0.004. More than a twenty-fold difference, and the latter barely fluctuates across seeds.

The comparison of mean error is just as clear: odometry 0.347, Gmapping 0.148, Cartographer 0.078 meters—that is, the particle filter cuts drift by about sixty percent, and submaps plus loop closure cut nearly half again on top of that. But more notable

still is the length of the error bars. Gmapping's endpoint error has a standard deviation of 0.168 meters, close to half its own mean, which is exactly the randomness of a particle filter: with the same parameters it is sometimes lucky and sometimes exhausts its particles. Cartographer's error bars are almost too short to see.

There is a structural reason behind this difference worth thinking through: Gmapping is a filter and can only move forward—history already resampled away cannot be corrected, so once endpoint error has accumulated it must be carried along; Cartographer keeps the whole pose graph, and the moment a loop closure appears the poses of all past submaps can be readjusted. Whether history can be rewritten is the most fundamental divide between the filtering and optimization routes, and the common source of every entry in Table 15-2.

An Honest Note About This Comparative Experiment

This is a teaching proxy implementation, not a performance benchmark of the Gmapping and Cartographer source code. Both were rewritten for this book's simulated site to bring out the architectural difference, and the performance of the actual packages depends heavily on parameters, sensors, and site.

The result does not hold under all conditions. If the site permits no loop (a corridor that runs straight to a dead end, say), Cartographer has no loop closure to use and its advantage narrows sharply—which is exactly what the third discussion question asks you to consider.

Particle count and submap count both change the result. This figure fixes 30 particles and 10 submaps, and you may sweep these two parameters yourself in Exercise 15-1.

Rerun this on the real packages in Part Two of Exercise 15-1 and compare your

measured numbers with this figure. Different numbers are normal; what matters is whether the trend agrees.

How to Choose

- A small site, teaching, understanding particle filters: Gmapping. Few parameters and intuitive concepts make it a good entry point.
- A large site, loop closure needed, a whole building to map: Cartographer. Its architecture was built for this.
- 3D modeling and backpack scanning: Cartographer 3D.
- Modern 2D SLAM in a ROS 2 environment: besides Cartographer, consider slam_toolbox—likewise based on graph optimization, with simpler parameters, and quite popular in the ROS 2 community in recent years.

What the Two Chapters Teach

Reading Chapters 14 and 15 side by side, you will see that they are the embodiments of the two theoretical routes of Chapters 11 (filtering) and 12 (optimization). Gmapping shows the ingenuity and the ceiling of the particle filter; Cartographer shows how graph optimization breaks through that ceiling. This is not merely a comparison of two tools but a miniature of more than a decade of technical evolution in SLAM.

You now command the two major systems of 2D SLAM. In the next chapter we turn the lens to the application closest to daily life—the cleaning robot. From random bump-and-turn to SLAM-based planning, from room segmentation to coverage path planning, we see how these theories operate in your own living room, and understand the real challenges that furniture, mirrors, and low obstacles pose in a home.

15.8 After Cartographer: The New Landscape of 2D SLAM, 2024–2026

Cartographer's paper appeared in 2016, which makes it, as this book is written, a system in its tenth year. Ten years is enough for a change of dynasty in robotics, yet Cartographer's core design—submaps, branch and bound, sparse pose adjustment—remains the standard answer in 2D LiDAR SLAM, and nearly every successor system makes local substitutions on that skeleton. This section surveys four main developments of the past two years so that you know where the frontier lies once this chapter is done.

1. A Shift in Maintenance: slam_toolbox Becomes the Practical Default in ROS 2

The largest change is not in the algorithm but in the ecosystem. As ROS 2 became mainstream, slam_toolbox, developed by Steve Macenski, gradually displaced Cartographer as the default 2D SLAM of most ROS 2 projects. It likewise takes the optimization route and likewise builds a pose graph by scan matching, but its parameters are far simpler than Cartographer's Lua configuration, and it has built in two capabilities in which Cartographer is weaker: map serialization and continued mapping (saving a completed pose graph and resuming from it at the next start-up), and a pure localization mode (loading an existing map to localize only, without updating it).

The implication for teaching cuts both ways. On one hand, if your goal is to get a project running on ROS 2, slam_toolbox is usually the path of least resistance; on the other, Cartographer remains the best teaching material for understanding why the design is as it is—its paper gives the clearest derivation of the branch-and-bound upper bound,

and many of slam_toolbox's design choices in fact presume a reader who already knows what Cartographer is doing. This book's advice: learn the principles with Cartographer, do the project with slam_toolbox.

2. Lifelong and Multi-Session Mapping: A Map Is No Longer a One-Off

The second development is an expansion of the problem statement itself. Traditional SLAM assumes one run, one map, and done; but a real site changes: furniture is moved, shelving rearranged, the season alters the lighting. Lifelong mapping addresses exactly this—how a robot walking the same site repeatedly can fold new observations into the old map while deleting obstacles that no longer exist, rather than letting the map grow ever dirtier. The serialization and resumption capabilities of slam_toolbox were designed with this in view.

Technically this turns the pose graph from a single trajectory into a collection of trajectories from several sessions, and loop closures occur not only within one run but between runs on different days (called multi-session SLAM). This line leads directly to the multi-robot collaboration of Part VI: several robots mapping at once and then merging their pose graphs is, in essence, the spatial version of multi-session SLAM.

3. Jointly Optimizing Pose and Map: The Occupancy-SLAM Line

The third is a theoretical challenge aimed directly at Cartographer's architecture. Cartographer's practice is to compute the pose by scan matching first and then insert the scan into the map—the pose is a variable and the map a by-product of it. Occupancy-SLAM, proposed by a team at the University of Technology Sydney (published at Robotics: Science and Systems 2024, with an improved version in 2025), argues that this division of labor is itself the bottleneck on accuracy: since the quality

of the map depends on the pose, correction of the pose should likewise be driven back by the consistency of the map, and so robot pose and the value of every occupancy cell should be treated as variables of one optimization problem and solved together.

The idea is mathematically natural; the difficulty is scale: a 2D occupancy grid easily runs to hundreds of thousands of cells, and treating each as a variable makes the problem several orders of magnitude larger than a traditional pose graph. The team's contribution is precisely in finding the sparse structure of this large problem so that it converges in acceptable time. If you have read the discussion of sparsity in Chapter 12 (Figure 12-3), you will recognize this as the same idea enlarged—sparsity is always the watershed of whether a large optimization can be practical.

4. Learning-Based Loop Closure Detection: From Geometric Matching to Scene Descriptors

The last development is in loop closure detection. Cartographer's branch and bound is a purely geometric method: it searches exhaustively over multi-resolution grids and accelerates by pruning with an upper bound. Its advantages are that it requires no training, depends on no dataset, and behaves entirely predictably; its drawback is that once the search range widens (the robot is picked up and set down on another floor, say—the kidnapping problem of Chapter 11) the computation still explodes.

The mainstream alternative in recent years is the scene descriptor: compress a scan into a fixed-length vector fingerprint, so that loop closure detection becomes a nearest-neighbor search in a fingerprint database, independent of how far the robot has traveled. The Scan Context family represents this route in 3D, and work in 2024 and 2025 extends it to loop closure from point cloud density images and to surveys that treat place recognition systematically as a problem in its own right. In 2D, limited by

how little information a single-beam scan carries, descriptors are less discriminative than in 3D, so branch and bound has not been displaced there—which also explains why the technical core of this chapter still repays close reading today.

Four Threads, One Common Theme

Engineering (slam_toolbox): packaging Cartographer's principles more usably, adding serialization and pure localization.

Problem statement (lifelong mapping): the map turns from a one-off product into an asset requiring long-term maintenance.

Mathematics (Occupancy-SLAM): drawing the map itself into the optimization variables, challenging the division of pose first, map second.

Perception (scene descriptors): replacing geometric exhaustive search in loop closure detection with feature retrieval, to handle large areas and the kidnapping problem.

The common theme: Cartographer's skeleton (submaps + pose graph + loop closure) has not been overturned; what is replaced is each component on that skeleton. Learn the skeleton and every later replacement of a part becomes readable.

Looking Back and Forward

This chapter has turned the graph optimization of Chapter 12 from a set of equations into a system that can be run down a corridor. Looking back at the arrangement of Part IV: Chapter 13 taught how to turn scans into a map, Chapter 14 untied the chicken-and-egg problem with a particle filter, and Chapter 15 demonstrated the other road—keep the history and allow it to be corrected. The numbers of Figure 15-4 show this

road's advantage at a site with a loop, and Section 15.8 states its boundary honestly: with no loop, loop closure has nothing to do.

From Chapter 16 the lens turns from what an algorithm looks like to how these algorithms survive inside a real product. The cleaning robot is the best first case: cheap, mass-produced, running every day in tens of millions of homes, and yet obliged to do everything this chapter discussed on an extremely small computing budget.

Formulations and Procedures

1. The Optimization of Local Scan Matching

When inserting a new scan into a submap, seek a pose ξ that makes the scan points fall as far as possible in places of high occupancy probability in the submap:

$$\xi^* = \operatorname{argmin}_{\xi} \sum_k \left(1 - M(T_\xi \cdot h_k)\right)^2$$

where h_k is the k-th point of the scan, T_ξ the pose transform carrying it into submap coordinates (Chapter 10), and $M(\cdot)$ the occupancy probability of that position in the probability grid. The expression means: make every scan point land where the map believes there is an obstacle. This is a nonlinear least-squares problem (Chapter 12), solved with Ceres.

2. Global Optimization (Sparse Pose Adjustment)

Global SLAM optimizes the submap poses Ξ^m and the scan poses Ξ^s to minimize the weighted residual of all constraints:

$$\operatorname{argmin} \tfrac{1}{2} \sum_{ij} \rho\left(E^2(\xi_i^m, \xi_j^s; \Sigma_{ij}, \xi_{ij})\right)$$

where E is the residual (the difference between the actual and the measured relation of the j-th scan to the i-th submap), Σ_{ij} the covariance of that constraint (its

confidence, Section 12.3), and ρ a robust kernel—which keeps a misjudged loop closure constraint from ruining the whole map (Section 15.4). This expression has exactly the form of the least-squares objective of Chapter 12, with the nodes replaced by submaps and scans.

3. Branch-and-Bound Search

The search space of global scan matching is defined as a three-dimensional window (translation x, translation y, rotation θ):

$$W = \{ -w_x, \ldots, w_x \} \times \{ -w_y, \ldots, w_y \} \times \{ -w_\theta, \ldots, w_\theta \}$$

Branch and bound proceeds by repeating branch—bound—prune:

Figure 15-5 The Branch-and-Bound Loop Closure Search

① Precompute multi-resolution grids (sizes 1, 4, 16, 64)

↓

② Starting at the lowest resolution, cut the search space into several large blocks (branch)

↓

③ For each block, compute an upper bound on the score on the low-resolution grid (bound)

↓

④ If a block's upper bound < the best score so far → prune the whole block and do not expand it (prune)

↓

⑤ Otherwise subdivide the block and repeat ②–④ at higher resolution

↓

⑥ On reaching full resolution with a score above the threshold → accept as a loop closure constraint

4. The Complete Data Flow of Cartographer

Figure 15-6 Cartographer End to End

Sensors: laser scans + IMU + (optionally) odometry

↓

[Local SLAM] scan matching → insert into the current submap → seal when full and open a new one

↓ (producing a local trajectory, which drifts)

[Global SLAM, a background thread]

Sample nodes → branch-and-bound matching against existing submaps → find loop closure constraints

↓

Form a pose graph (submap poses + scan poses)

↓

Every few seconds, run sparse pose adjustment with Ceres (including a robust kernel)

↓

Output: a globally consistent map and trajectory (the whole map straightened at a loop closure)

Case Studies

Case 15-1 The Moment of Loop Closure: The Most Striking Three Seconds in Class

Case 12-1 of Chapter 12 described the picture of a loop closure straightening a map, and Cartographer makes that picture an experiment reproducible in class.

The typical teaching scene runs like this: students teleoperate the robot once around the department building. All goes well for the first half and the map is neat; by the second half the wall lines begin to diverge from what was built at the start, obvious ghosting appears in the corridor, and puzzlement spreads across the students' faces. Then, the instant the robot returns near its starting point—the background thread finds the loop closure constraint and global optimization runs—the whole map reassembles on the screen in a flash, the ghosts merge into one, the offset corridor joins up, and the trajectory forms a beautifully closed loop.

Those three seconds condense the graph optimization of Chapter 12 and the branch and bound and sparse pose adjustment of Section 15.4 into a single visible instant. Teaching experience shows that students who have seen this with their own eyes understand why graph optimization is used far better than any lecture can convey. Instructors are advised to arrange enough space for the robot to complete a full loop—the class time in this chapter most worth investing.

Case 15-2 Why Google Built Cartographer: Ambitions for Indoor Maps

Google did not develop Cartographer out of purely academic interest. By then Google had mapped the world's roads with Street View cars, but indoor spaces—malls, airports, stations, museums—remained blanks on the map. Traditional surveying was too slow and too expensive for indoor mapping, and a technology that could map while walking was required.

This explains several of Cartographer's design orientations: why it must be real time (the operator needs to know at once what has not yet been scanned), why it must handle large scenes (a whole mall), why a backpack system was built (anywhere a

person can walk must be scannable), and why loop closure matters so much (walking a large area must accumulate drift).

This case offers an important lesson: a technology's design orientation is often decided by application requirements. Gmapping came out of an academic laboratory and pursued algorithmic elegance; Cartographer came out of industrial need and pursued practicality at large sites. Neither is better—only more or less suitable, which confirms once again the idea of Chapter 8 that sensor selection is a combined judgment.

Case 15-3 Experiments in Sensor Configuration: What Configuration Suits What Site

One research group tested systematically how different sensor combinations affect Cartographer's performance, with instructive results: in a cluttered room, three 2D LiDARs with an IMU and odometry gave the lowest error; in a long corridor, a single 3D Velodyne LiDAR with an IMU performed best.

The result agrees entirely with earlier discussions in this book. In a cluttered room, several 2D LiDARs scanning from different angles cover more geometric features and reduce occlusion (the height blind spot of Chapter 8); the difficulty of a long corridor is a lack of longitudinal constraint (Section 13.6), and a 3D LiDAR can see the structural variation of ceiling and floor, filling exactly that gap.

The lesson for university projects and undergraduate practicals is: do not chase the most expensive sensor blindly, but choose the configuration by the character of the site. This also makes an excellent project topic—measure the mapping quality of different sensor combinations at various campus sites (classroom, corridor, atrium) and

compare quantitatively by the methods of Section 13.7. Such a study has both teaching value and practical significance.

Hands-On Activities

Exercise 15-1 Cartographer Mapping and Watching the Loop Closure

Purpose: to run 2D Cartographer mapping, watch submaps form and the instant a loop closure straightens the map, and run a comparative experiment against the Gmapping of Chapter 14. This is the most important exercise in Part IV.

Part One: Mapping in Simulation and Watching the Loop Closure (the whole class)

1. Install and launch Cartographer's 2D configuration in the TurtleBot3 Gazebo simulation environment (Chapter 28).
2. Display the submaps and the trajectory in RViz, move within a small area first, and observe how submaps form one by one and partly overlap.
3. Teleoperate the robot once around a complete loop of the simulated world, watching throughout: the map is neat in the first half, and misalignment begins to appear in the second.
4. The key moment: on returning near the starting point, watch for whether the loop closure triggers and the map is straightened. Record this instant and replay it for discussion in class.
5. If no loop closure triggers, try adjusting the loop closure score threshold and the sampling ratio (Table 15-1) and run again.

Part Two: The Comparative Experiment—Cartographer versus Gmapping (the core)

6. In the same simulated world and along the same route, map once with Gmapping (Chapter 14) and once with Cartographer.
7. Assess both maps by the methods of Section 13.7: are the walls straight? Does the loop close? Is there ghosting? Measure key dimensions on the map against the simulation settings.
8. Place the two maps side by side, mark the regions where they differ most, and explain why (hint: Table 15-2).
9. Observe resource use: compare the CPU and memory consumption of the two against the discussion of formulation 5 in Chapter 14 and Section 15.2.

Part Three: Mapping on a Physical Platform (for classes with hardware)

10. Prepare the Lua configuration file and adjust it to your platform (IMU or not, odometry or not, LiDAR model and mounting position).
11. Map the classroom first to verify the configuration, then take on a larger site—ideally a corridor or a floor of the department building that permits a complete loop.
12. Walk slowly and be sure to complete the whole loop back to the starting point, watching for the loop closure.
13. Export the map and assess its quality, comparing with the Gmapping result at the same site.

Discussion Questions

1. Did you observe the instant the loop closure straightened the map? Which areas of the map changed most before and after, and why those?
2. In the comparative experiment, in what respects did the Cartographer and Gmapping maps differ most? Which entries of Table 15-2 does this confirm?

3. If the site has no loop (merely a corridor that comes to a dead end, say), can Cartographer's loop closure mechanism still do anything? Does it still have an advantage over Gmapping then?
4. (Undergraduate) Section 15.3 noted that too low a loop closure sampling ratio misses loop closures while too high a ratio slows real-time operation. Adjust this parameter in practice, record how loop closure detection rate and system latency change, and find a suitable value for your site.

Extended Challenge (Undergraduate)

First: design a false loop closure experiment—build two rooms of nearly identical appearance in the simulated world and observe whether Cartographer misjudges a loop closure; if it does, observe whether the robust kernel lessens the severity. This will deepen your understanding of the robustification of Section 15.4. Second: if your school has a 3D LiDAR, try Cartographer's 3D mode by scanning a laboratory or a flight of stairs to produce a three-dimensional point cloud model, experiencing the idea of backpack modeling (Section 15.6) and preparing for the 3D SLAM of Part V (Chapters 17 and 18).

Further Reading

1. W. Hess, D. Kohler, H. Rapp, and D. Andor, "Real-Time Loop Closure in 2D LIDAR SLAM," ICRA, 2016. (The original Cartographer paper and the principal basis of this chapter.)
2. The official Cartographer documentation (google-cartographer.readthedocs.io) and the Cartographer ROS documentation. (Includes the "Algorithm walkthrough for tuning" guide; essential for implementation.)

3. Google Open Source Blog, the "Cartographer" release announcement. (Explains the development motivation and the background of indoor surveying.)
4. J. Elseberg and A. Nüchter, "Improving Google's Cartographer 3D Mapping by Continuous-Time SLAM," ISPRS Archives, 2017. (Research improving the 3D mode, with measurements from a backpack system.)
5. The slam_toolbox open-source project (GitHub). (Another graph-optimization 2D SLAM in ROS 2, useful for comparison with Cartographer.)
6. The official Ceres Solver documentation (ceres-solver.org). (The optimization library Cartographer uses, echoing Chapter 12.)
7. Cyrill Stachniss, the Graph-Based SLAM course videos (YouTube). (The theoretical basis of graph optimization SLAM; good read alongside this chapter.)

Summary

1. Cartographer was developed by Google and open-sourced in 2016, its core feature being real-time loop closure; it divides into two concurrently running subsystems: local SLAM (the front end, building submaps that are locally consistent but drift) and global SLAM (the back end, seeking loop closure constraints in the background and optimizing the pose graph).
2. A submap is a small probability grid map made of a few dozen scans; the three reasons for submaps are bounding the accumulation of error, providing a unit for optimization (the node count falls sharply, so real-time operation is possible), and serving as a stable object for loop closure comparison. Memory is proportional to the size of the scene, so a whole building can be handled.

3. The system has two kinds of scan matching: local matching inserts a new scan into the current submap (solving a nonlinear least-squares problem with Ceres); global matching searches for loop closures, is far harder, and must limit the number of sampled nodes to balance loop closure success against real-time operation.
4. Branch and bound is the key to real-time loop closure: on precomputed multi-resolution grids it computes an upper bound on the score at low resolution and prunes whole regions that cannot win, cutting the search sharply; once constraints are found, sparse pose adjustment straightens the whole map, and a robust kernel prevents a false loop closure from causing catastrophe.
5. A 2D implementation requires laser, TF, and (preferably) an IMU; Cartographer is configured by Lua files with a good many parameters, the key ones being the submap scan count, the resolution, the loop closure sampling ratio, and the score threshold; be sure to complete the whole loop to observe loop closure.
6. 3D mode requires a 3D LiDAR and an IMU, with voxel-grid submaps and a heavy computational load; backpack spatial modeling lets an operator build a three-dimensional model of a whole building while walking, applied to BIM, heritage preservation, and facility management—a project direction of great potential at a university site.
7. Compared with Gmapping: Cartographer is based on graph optimization rather than a particle filter, on submaps rather than one map per particle, with loop closure as a core feature, memory proportional to the scene, and support for 3D and the IMU, though with more complex parameters. The two are the embodiments of the two theoretical routes of Chapters 11 and 12.

The two major technical protagonists of Part IV are now complete. In the next chapter we take these techniques back to the setting closest to daily life—the cleaning robot. From random bump-and-turn to SLAM-based planned cleaning, from room segmentation and keep-out zones to coverage path planning, and facing honestly the real challenges of a home: chair and table legs, mirrors, low obstacles, and pets. You will find that every theory of this part is quietly at work in your own living room.

A Taiwan-Authored University Textbook

@prof

Simultaneous Localization and Mapping

Simultaneous Localization and Mapping, SLAM

Chapter 16

SLAM in the Cleaning Robot

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

This chapter closes Part IV and is the first time in the book that the technology accumulated over fifteen chapters is placed as a set inside a product you can buy and hold. The previous three chapters were about the algorithms themselves—how an occupancy grid is built, how particles are filtered, how a pose graph is solved; this chapter asks another question: what happens when those algorithms must run on a machine costing twenty thousand NT dollars, with a computing chip no better than a phone's, working alone for eight hours a day in an unfamiliar home? The answer is usually not that the algorithm is not good enough but that the real world is far more complicated than the corridor in a paper.

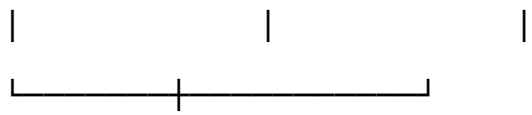
So this chapter should be read differently from the previous three. Those built tools from the bottom up; this one dissects a product from the top down: which tools it uses, where it compromises, and how it makes up for them. Having read it, look back at the angular resolution of Chapter 8, the map symptoms of Chapter 13, and the slip problem of Chapter 14, and you will find that every limitation that seemed abstract at the time corresponds to some concrete behavior of the cleaning robot in your home.

Figure 16-0 The Connection Map of Chapter 16

The first three parts: sensing and mathematical tools

Chapter 3 Navigation architecture Chapter 8 2D LiDAR Chapter 11 Probabilistic

estimation



▼

Part IV: Chapter 13 Occupancy grid → Chapter 14 Gmapping

→ Chapter 15 Cartographer

▼

[Chapter 16 The cleaning robot: where technology meets the ground]

Coverage path planning · room segmentation · keep-out zones · sensor fusion

|

▼

Part V 3D SLAM Part VIII Navigation planning Part IX Projects

Learning Objectives

The theory of Part IV comes at last back to the setting closest to daily life—your own living room. The cleaning robot is the most successful popularization of SLAM technology, and nearly every idea of the preceding fifteen chapters has its counterpart in this small disc of a machine. This chapter closes Part IV and is a lesson in bringing theory home. After completing this chapter, you should be able to:

1. Explain why a cleaning robot needs SLAM, and what the consequences are without it.
2. Compare random bump-and-turn cleaning with SLAM-based planned cleaning, and understand the quantitative meaning of coverage and efficiency.
3. Explain how a LiDAR cleaning robot builds a map of a home, and connect it to the techniques of Chapters 13 through 15.

4. Explain the principles of room segmentation, keep-out zones, and coverage path planning.
5. List the sensor combination of a cleaning robot and describe the role of each and how they complement one another.
6. Analyze the real challenges of a home: chair and table legs, mirrors, low obstacles, and a dynamic environment.
7. Understand the basic strategies and evaluation metrics of coverage path planning from a cleaning route.

Core Concepts

Chinese Term	English Term	Explanation
覆蓋路徑規劃	Coverage Path Planning	Planning a path that visits every traversable position in a region.
弓字形路徑	Boustrophedon Path	A back-and-forth cleaning route like plowing a field; the basic form of coverage planning.
隨機碰撞式	Random Bounce	The strategy of early cleaning robots: turn on hitting an obstacle, with no map and no planning.
覆蓋率	Coverage Rate	The proportion of the cleanable area actually cleaned.
重複率	Redundancy Rate	The proportion of positions cleaned more than once; the lower the more efficient.
房間分割	Room Segmentation	Automatically dividing a whole map into meaningful room blocks.
禁區	No-go Zone	An area the user marks on the map that the robot may not enter.

Chinese Term	English Term	Explanation
虛擬牆	Virtual Wall	A virtual boundary on the map, functioning much like a keep-out zone.
懸崖感測器	Cliff Sensor	An infrared sensor detecting the floor ahead suddenly ending (stairs).
3D 飛時測距	3D ToF	A sensor measuring the shape of obstacles ahead three-dimensionally.
結構光	Structured Light	Projects a light pattern to read the shape and distance of obstacles.
多樓層地圖	Multi-floor Map	Remembers the maps of several floors and recognizes the current floor automatically.

16.1 Why a Cleaning Robot Needs SLAM

The question seems simple at first: is it not just a matter of sweeping the floor clean? But think it through, and sweeping the whole floor of a home once without repeating too often requires answering the three navigation questions of Chapter 3: where am I? Where am I going? How do I get there?

Three Difficulties Without SLAM

- Not knowing what has been cleaned: without a map and localization the robot cannot record that it has already covered a patch. The result is that some places are cleaned five times and others not at all.
- Not knowing when it is finished: without a map there is no knowing how large the home is or how much remains. Early models could rely only on a timer—thirty minutes and knock off, regardless of whether anything was finished.
- Not knowing the way home: when the battery runs low it must return to the dock, and without a map it can only wander and hope, often stranding itself halfway.

Four Capabilities SLAM Brings

1. Systematic cleaning: only with a map can a boustrophedon route be planned to cover zone by zone, raising coverage sharply and lowering repetition (Section 16.2).
2. Resuming after a break: when the battery runs low it docks, and after charging it can return to where it left off—which requires accurate localization and a remembered map.

3. User interaction: showing the room map in an app lets the user say "clean the kitchen only," set keep-out zones, and point to where it should clean. This is the product value a map brings.
4. Consistency across cleanings: a second cleaning need not explore afresh and can simply load the existing map and localize (the AMCL of Case 14-3 in Chapter 14).

Threading the Earlier Chapters Together

The cleaning robot is almost a physical review of the first fifteen chapters of this book: it senses the environment with the 2D LiDAR of Chapter 8, records space in the occupancy grid map of Chapter 13, localizes and maps at once with the SLAM algorithms of Chapter 14 or 15, integrates sensors with the coordinate transforms of Chapter 10, and relocalizes with the particle filter of Chapter 11 (the kidnapping problem of Chapter 3). To understand the cleaning robot is to understand the whole of Part IV.

16.2 Random Bump-and-Turn Cleaning versus SLAM-Based Planned Cleaning

The evolution of the cleaning robot is a miniature of the move from reactive to deliberative (the architectural discussion of Chapter 3).

Random Bump-and-Turn: Reactive Architecture Taken to the Limit

When iRobot launched the first-generation Roomba in 2002, it used a random bump-and-turn strategy: drive straight, turn through a random angle on hitting something, and keep going, alternating with a few simple behaviors such as spiraling and wall-following.

The inspiration for this design in fact comes from the behavior-based architecture mentioned in Chapter 3—no model, no planning, perception driving behavior directly. Its advantages are that it is extremely cheap, the program is simple, and it cannot fail because of a wrong map; its drawbacks are equally plain: a disorderly route, uneven coverage, and a very long time.

Interestingly, the strategy is not mathematically baseless: given long enough, a random walk will in theory cover any connected region. The trouble is that long enough may mean an hour—and there is never any guarantee that cleaning has actually finished.

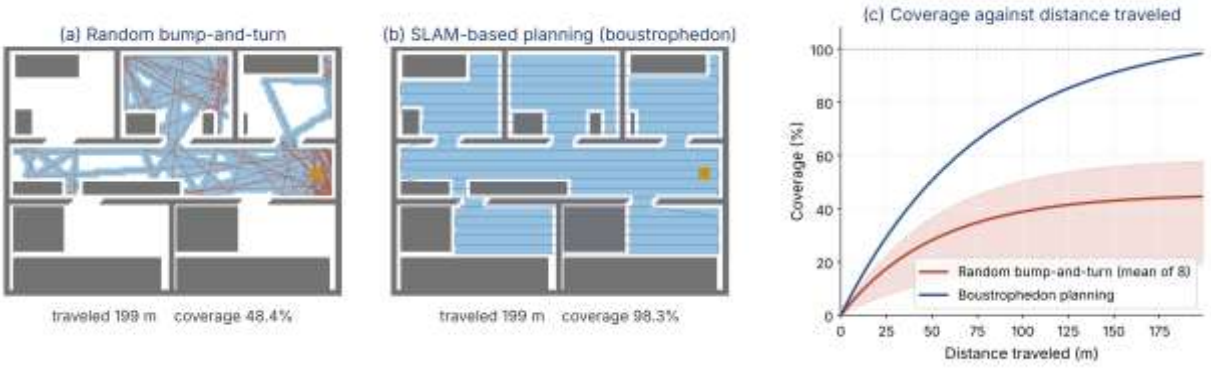
SLAM-Based Planning: The Victory of Deliberative Architecture

In 2010 Neato first fitted a 2D LiDAR to a cleaning robot, letting it build an occupancy grid map and plan a systematic route. Modern models are no longer merely rotating brushes steered by random bumps but run on-device AI to build maps, avoid obstacles, and adapt to real domestic life.

The core of planned cleaning is the boustrophedon path (the word means "as the ox turns in plowing"): the robot cleans in a straight line to the end in one direction, turns 180 degrees, moves over by one brush width, and sweeps back the other way, repeating to cover the whole area.

Figure 16-1 Same Apartment, Same 199 Meters: The Results of Two Cleaning

Strategies



This figure turns the verbal differences of Table 16-1 into measurable numbers through one comparative experiment. The scene is a three-bedroom apartment of 9.0 × 6.6 meters with a master bedroom, a second bedroom, a study, a living room, a kitchen, and a hallway, furnished with a bed, a sofa, a desk, and a counter; subtracting the wall and furniture margins the robot's radius cannot enter, the cleanable area is 27.5 square meters. Both strategies start from the dock on the right (the gold square) and both travel 199 meters—that is, battery consumption is the same.

Panel (a) is random bump-and-turn: drive straight and turn through a random angle on contact. The path shows the classic tangle, and the result is only 48.4% covered. More notable is where it covered—the master bedroom, the hallway, and the kitchen are clearly covered while the second bedroom, the study, and the living room are nearly blank. The reason is concrete: the doorways are only 85 centimeters wide, leaving a passable gap under 60 centimeters after the robot's radius, and the probability that a random heading passes exactly through that gap is low. A random strategy does not "get there eventually"; it gets trapped in the first few rooms it happens to enter.

Panel (b) is SLAM-based planning: the map is first used to cut the space into rooms (Figure 16-2 in the next section), then a boustrophedon path is run in each room with the shortest connections between them. The same 199 meters gives 98.3% coverage.

The neat rows of back-and-forth lines on the path plot are the origin of the name boustrophedon; the unswept white margin around the furniture is the physical limit imposed by the robot's radius—which also explains why 100% coverage is hard to reach in practice.

Panel (c) plots coverage against distance traveled and is the most informative panel in the figure. The solid red line is the mean of eight random trials, and the pale red band the range from best to worst. The two curves nearly coincide over the first 25 meters—reasonably enough, since at the start there are unswept places everywhere and even wandering pays. But from about 40 meters they diverge: the boustrophedon keeps climbing linearly toward 100% while the random strategy quickly flattens, with final coverage across the eight trials falling between 19.5% and 59.0%.

The width of that pale red band is more worth discussing than the mean itself. It represents the fact that with the same machine, the same program, and the same time, today's result and tomorrow's may differ threefold. For an appliance used every day, unpredictability is more fatal than low efficiency—the user cannot tell whether cleaning has finished. This is the quantitative version of the difficulty of "not knowing when it is finished" in Section 16.1, and the greatest value SLAM brings to the cleaning robot: not merely cleaning better but cleaning predictably.

A Note About This Simulation

This is a simplified simulation for teaching: the robot is treated as a circle 26 centimeters in diameter with an effective brush width of 26 centimeters, excluding the extra coverage of a side brush.

The random strategy is implemented as drive straight, turn through a random

angle on contact. A real Roomba also interleaves spiral and wall-following modes and would achieve somewhat higher coverage than this figure, but the structural problem of being trapped in a multi-room home remains the same.

The boustrophedon plan traveled 199 meters, and the theoretical shortest covering distance is 27.5 ÷ 0.26 ≈ 105.6 meters, so the repetition ratio is about 1.88. That figure is middling by this chapter's metrics—turns, detours around furniture, and travel between rooms are all included.

You can reproduce this figure yourself in Part Three of Exercise 16-1 and try to press the repetition ratio below 1.5.

Item of Comparison	Random bump-and-turn	SLAM-based planning
Builds a map?	No	Yes (an occupancy grid map)
Cleaning route	Random and disorderly	Boustrophedon and systematic
Coverage	Not guaranteed; depends on time	High; can approach complete coverage
Repetition ratio	High	Low
Time required	Long	Short
Can resume after a break?	No	Yes
Cost	Very low	Higher (a LiDAR or camera is needed)
Tolerance of map error	Unaffected	A wrong map means wrong cleaning

Table 16-1 A Comparison of the Two Cleaning Strategies

Note the last row: planning is not without drawbacks. If SLAM mapping fails (the map symptoms of Section 13.7), the robot cleans according to a wrong map and may miss a whole room instead. This echoes the conclusion of Chapter 13: map quality directly decides whether the system succeeds.

16.3 The LiDAR Cleaning Robot and 2D LiDAR Mapping

Case 1-1 of Chapter 1 noted that in 2017 Xiaomi launched a LiDAR cleaning robot at an affordable price, bringing SLAM technology into ordinary homes in quantity for the first time. This section looks closely at how it works.

Hardware Configuration

A typical LiDAR cleaning robot has a raised round turret on top containing a rotating 2D LiDAR (Chapter 8), scanning at about the height of the body's top (8 to 10 centimeters above the floor). It rotates several times a second and measures ranges in hundreds of directions around it.

Cost control is worth noting: the LiDAR in a domestic cleaning robot mostly uses low-cost triangulation ranging, with a range of about 6 to 8 meters and accuracy of a few centimeters—far short of industrial models but ample for indoor mapping. This is an instance of the Chapter 8 principle that resolution must match the application requirement.

The Mapping Procedure: Chapters 13 through 15 in Practice

5. Entering an unfamiliar environment for the first time: the robot runs SLAM, building an occupancy grid map and estimating its own pose as it moves

(Chapter 13). Most models first follow the walls around once to establish the outline of the home, then deal with the interior.

6. Choice of algorithm: early models mostly used particle filter methods (Chapter 14); in recent years, as computing chips have improved, more and more use graph optimization with loop closure detection (Chapter 15), making maps of larger homes more consistent.
7. Map storage: the completed map is stored in the machine and in the cloud, and loaded directly for later cleanings.
8. Later cleanings: after loading the map it localizes (AMCL-type methods, Chapter 11) without remapping, and can plan routes directly.

A Recent Trend: LiDAR and Vision Together

Modern premium models no longer rely on LiDAR alone. The industry's practice is for the laser LiDAR to build the map and track the robot's position while an AI camera handles the visual SLAM side—recognizing objects on the floor and deciding what to do about them. This is the commercial practice of "LiDAR gives geometry, the camera gives semantics" from Chapter 9.

This combination answers directly to the complementarity analysis of Table 8-2 in Chapter 8: LiDAR does not fear the dark and is geometrically precise but does not know what a thing is; a camera can recognize objects but fails in dim light and on untextured surfaces. Some manufacturers therefore add an LED beside the camera to preserve vision in dark or low spaces.

16.4 Room Segmentation, Keep-Out Zones, and Coverage

Having a map is only the first step. Three further functions are needed to make the

map genuinely useful, and together they constitute what the user perceives as intelligence.

1. Room Segmentation

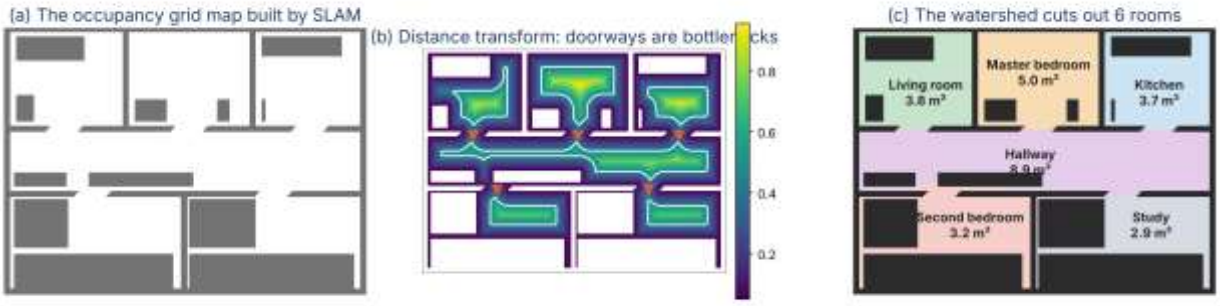
Room segmentation automatically divides a whole occupancy grid map into meaningful blocks such as living room, bedroom, and kitchen. This is crucial to the user experience—without segmentation the user can only say "clean everything"; with it, the user can say "clean the kitchen only."

The basic idea of segmentation is to find the bottlenecks of the space: at doorways and narrow passages the width of free space narrows abruptly, and these positions are natural boundaries. Common methods include using a distance transform to find the core region of each room and then expanding outward, or detecting the geometric features of door frames.

Having built a map, modern models can label rooms, exclude keep-out zones, and even remember which areas need cleaning more often. This already touches on the semantic map idea mentioned in Chapter 3—the robot knows not only that a place is traversable but that it is the kitchen. Chapter 32 discusses semantic labeling with AI in full.

Figure 16-2 Room Segmentation: Finding the Doorway Bottleneck with a Distance

Transform



Room segmentation sounds like something requiring artificial intelligence—after all, which room is the bedroom is a semantic question. But this figure shows that two steps of purely geometric computation do the job rather well. Panel (a) is the occupancy grid map built by SLAM, at this point merely a collection of black and white cells with no notion of a room.

Panel (b) is the key step: the distance transform. It computes for every free cell how far it is from the nearest wall, the brighter the color the more open. One pattern is immediately visible—the center of a room is bright (far from walls) while a doorway is a dark narrow slot (walls on both sides, near to both). The five red triangles mark this apartment's doorways, and all of them appear as bottlenecks in the distance transform. The white contour marks the threshold of 0.45 meters: the doorway values are about 0.42 meters and are just cut off, so the cores of the rooms separate naturally into distinct blocks.

Panel (c) is the second step: using those cores as seeds for a watershed expansion, letting each core grow back along the gradient of the distance transform until they meet at the doorways. The result is six blocks corresponding exactly to the master bedroom, second bedroom, study, hallway, living room, and kitchen, with their areas computed as well. Not a scrap of machine learning was used in the process—only the

distance transform and the watershed, each one line of code in OpenCV.

The product significance of this result is direct: with segmentation the app can display "living room, 3.8 square meters" and let the user tap "clean the kitchen only"; with segmentation, coverage planning can proceed room by room (which is exactly what Figure 16-1(b) does) rather than cutting long strips across the whole plane. But it must be said honestly that a purely geometric method has its limits: where an open kitchen flows into the living room there is no door and thus no bottleneck to find, and such layouts often get merged into one block or cut in the wrong place—which is exactly why modern models have begun to bring in semantic information.

How to Choose the Distance Transform Threshold

The physical meaning of the threshold is that a room's core must be at least this far from a wall. It must be greater than half the doorway width and less than half the room width.

In this figure the doorway is 0.85 meters wide (half-width about 0.42) and the narrowest room has a half-width above about 0.6 meters, so 0.45 meters is a safe middle value.

Threshold too low: the doorway is not cut and adjacent rooms merge into one block (which happens below 0.40 in this chapter's simulation).

Threshold too high: a room's core is fragmented by furniture and one room splits into several blocks, which must then be merged by a minimum-area filter.

Doorway widths are fairly standard in practice (80 to 90 centimeters in most Taiwanese homes), so this threshold can be hard-coded; what is genuinely hard are open-plan layouts and doorless passages.

2. Keep-Out Zones and Virtual Walls

A keep-out zone lets the user mark on the app map an area the robot may not enter—the pet's bowl, the child's block area, a tangle of cables. A virtual wall draws a line that may not be crossed.

Technically this is adding artificial obstacles to the costmap (Chapter 3): in path planning the keep-out zone is treated as impassable. It is a clever design—nothing in SLAM or the sensors need change, only a layer of user-defined constraints added at the planning level. You might consider that this is in fact a way of injecting the user's knowledge into a robotic system.

3. Coverage Path Planning

Coverage path planning (CPP) differs essentially from ordinary path planning:

- Ordinary path planning (Chapters 3 and 29): find the shortest path from A to B, the goal being arrival.
- Coverage path planning: find a path that visits every traversable position, the goal being to miss nothing.

The cleaning robot is the archetypal application of the latter. The basic strategy is to divide the map into obstacle-free strips, run a boustrophedon path within each, and connect the strips in order. The robot uses its onboard sensors and (where available) a preloaded floor plan outline to plan a coverage path reaching every accessible room, corridor, and space.

Evaluation Metrics

- Coverage: the area actually cleaned divided by the cleanable area. The ideal is near 100%, but under furniture and in corners complete coverage is often hard.

- Repetition ratio: total distance traveled divided by the theoretical shortest covering distance. The closer to 1 the more efficient.
- Cleaning time: the time to finish the whole home, the metric the user feels most directly.

These three metrics pull against one another: raising coverage usually raises repetition and time. The engineering trade-off is the core of product design. It is also a quantitative line of thinking the project grading rubric of Chapter 31 may draw on.

16.5 A Cleaning Robot's Sensors: LiDAR, IMU, Bumper, Cliff Sensors

The sensors on a cleaning robot are almost a condensed version of Part II of this book. This section describes the role of each and how they complement one another.

Sensor	Role	Corresponding Chapter
2D LiDAR	The mainstay of mapping and localization, measuring ranges all round	Chapter 8
IMU	Detects turning and tilt, assisting pose estimation	Chapter 7
Wheel encoders	Provide odometry, the source of SLAM's motion prediction	Chapters 3 and 13
Bumper	The last line of defense when the LiDAR has not seen something	This chapter
Cliff sensors	Downward infrared, preventing a fall down the stairs	This chapter
Infrared /	Close-range obstacle detection, filling	Chapter 8

Sensor	Role	Corresponding Chapter
ultrasonic	the LiDAR's blind spot	
AI camera	Recognizes objects and semantics, avoiding cables and pet accidents	Chapters 9 and 32
3D ToF / structured light	Measures the three-dimensional shape of obstacles ahead	Chapters 8 and 9

Table 16-2 The Sensor Suite of a Cleaning Robot

Why So Many Sensors Are Needed

The answer is the height blind spot of Case 8-1 in Chapter 8: a 2D LiDAR scans one horizontal slice and cannot see anything below or above it. Therefore:

- Low obstacles (cables, slippers, pet toys): below the LiDAR's scan plane and entirely invisible, so forward-facing infrared, 3D ToF, or a camera is required.
- Overhanging obstacles (hanging curtains, the edges of low furniture): may sit just above or below the scan plane, with unstable detection.
- Stairs and drop-offs: a horizontal LiDAR scan cannot detect the floor suddenly ending at all, so downward cliff sensors are required—an indispensable safety device on a cleaning robot.
- Glass and mirrors: the old problem of Chapter 8, with the laser passing through or reflecting away, so the bumper must serve as the backstop.

Figure 16-3 A Height Profile: The Things a 2D LiDAR Cannot See



Table 16-2 lists eight sensors, and a beginner's first question is usually: what are so many of them for? This side-view profile is the answer. The red dashed line is the 2D LiDAR's scan plane, about 9 centimeters above the floor—note that it is a plane, not a field of view. The LiDAR knows everything on that plane and is wholly ignorant of everything off it. Chapter 8 of this book called it the height blind spot, and a home is where the height blind spot is most fatal.

Reading left to right, four typical failures. Cables and slippers are only three or four centimeters high and fall entirely below the scan plane; the LiDAR does not merely see them poorly but has no idea they exist—the physical reason a cleaning robot tangles itself in cables and drags a sock through the whole house. A dining chair is a different failure: wide above and narrow below, the scan plane cuts only the slender legs, and the angular resolution formula of Chapter 8 tells us that at sufficient distance the spacing of adjacent beams exceeds the leg's width and the whole leg falls between two beams and is missed. Case 16-2, "why your cleaning robot always hits that chair," is about exactly this geometric fact.

Floor-to-ceiling windows and mirrors are the optical difficulty stressed repeatedly in Chapter 8: the laser either passes straight through or is reflected elsewhere, producing either a room on the map that does not exist at all or an invisible wall. Stairs and drop-

offs are more dangerous still—a horizontal scan cannot detect the floor suddenly ending, and without another sensor the robot would walk straight down the stairs.

The three colored regions in the figure are the answer that fills the gaps, and their placement itself explains the design logic: the 3D ToF and the AI camera open a conical field of view forward and upward, covering the three-dimensional shape above and below the scan plane; the cliff sensors point straight down, covering the floor itself; and the bumper sits on the front edge of the body as the last line of defense—when every other sensor has failed, at least physical contact remains. These three layers, from far to near and from precise to coarse, are the most everyday instance of the sensor fusion idea of Part II: there is no all-powerful sensor, only an appropriate division of labor.

An Observation You Can Make Yourself

Lay a book flat on the floor and see whether your cleaning robot avoids it—a book is about 3 centimeters thick, below the scan plane.

Replace it with a cardboard box (more than 10 centimeters tall) and it will usually be skirted cleanly—because it has entered the scan plane.

This comparison needs only two things already in the house, yet it lets you confirm with your own eyes that the scan plane is a real line.

A further observation: replace the book with a slender chair leg or the base of a floor lamp and place it at various distances to test the effect of the angular resolution of Chapter 8.

Modern Premium Configurations

The sensor suites of recent premium models have grown richer. Some models pair a single AI camera with 3D structured light to form an advanced obstacle detection

system that recognizes a great many common obstacle types. The sensors each have their job: an LED keeps the camera's vision in dark or low spaces, 3D structured light reads the shape of obstacles directly ahead, and 3D ToF measures height and width at once while holding up in strong light.

This evolution confirms again the core conviction of this book: there is no all-powerful sensor, only an appropriate combination. The cleaning robot is the most everyday teaching material for sensor fusion (Part II).

16.6 SLAM Challenges in a Home: Furniture, Mirrors, Low Obstacles

A laboratory classroom is a friendly scene for SLAM (Section 13.6), but a home is another matter. This section faces the challenges of a real home honestly—the part of the chapter you can most readily confirm by watching your own cleaning robot.

1. Slender Chair and Table Legs

Case 8-1 of Chapter 8 discussed this in detail: a dining chair's legs are slender and at a distance may fall entirely between two beams and be missed (the angular resolution formula of Chapter 8). The result is that the leg is not on the map and the robot drives straight into it.

- Remedy: at close range the angular resolution suffices, with the bumper as a backstop; modern models use 3D ToF or a camera to fill in detection at close range.

2. Mirrors and Floor-to-Ceiling Windows

A mirror lets the LiDAR "see" the virtual image within it and draw on the map a room that does not exist at all; a floor-to-ceiling window lets the laser pass straight

through and the robot believes it can go there. This is the LiDAR weakness stressed repeatedly in Chapter 8, and it is extremely common in homes.

- Remedy: have the user set a keep-out zone by hand in the app (Section 16.4); some models detect glass with ultrasound (sound waves reflect off glass); and cover mirrors temporarily during mapping.

3. Low and Dynamic Obstacles

Cables, socks, pet toys, and pet accidents on the floor are the cleaning robot's most famous sources of disaster. They are below the LiDAR's scan plane and in a different place every day—they will not be on the map.

- Remedy: this is exactly where an AI camera earns its keep. The sensing system fuses data from LiDAR, camera, and sometimes a structured-light sensor to detect walls, furniture legs, toys, and even cables, and it does so continuously.

4. A Dynamic Environment: Homes Change

A chair is pulled out, a suitcase is left in the living room, a door opens and closes—a home changes every day. This is a severe test for SLAM, because the map must both remember permanent structure (walls) and forget temporary objects (the map plasticity of Section 13.5).

- Remedy: clamp the occupancy probability to keep the map plastic (Section 13.5); distinguish a permanent map (walls and furniture) from a temporary obstacle layer (objects detected during this cleaning); and some models tally over several cleanings which obstacles are permanent.

5. Changes in Floor Material

Going from tile to carpet changes the friction and slip characteristics of the wheels

and degrades odometry (the slip problem of Section 14.6); carpet also lets the robot sink in, changing the body height and so the LiDAR's scan plane.

- Remedy: raise the noise assumed for odometry in SLAM and rely more on scan matching; some models detect carpet and adjust behavior (raising suction, lifting the mop).

Challenge in the Home	Root Cause	Principal Remedy
Slender chair and table legs	Insufficient angular resolution; missed at a distance	Close-range detection, bumper, 3D ToF
Mirrors and floor-to-ceiling windows	The laser passes through or reflects, producing a virtual image	User-set keep-out zone, ultrasound, temporary covering
Low dynamic obstacles	Below the LiDAR's scan plane	AI camera recognition, 3D structured light
The environment changes daily	The map must both remember and forget	Clamp the probability, layer the map, tally over several runs
Carpet and changes of floor	Degraded odometry, changed body height	Raise the odometry noise, rely on scan matching

Table 16-3 SLAM Challenges in a Home and Their Remedies

16.7 Teaching Activity: Understanding Coverage Path Planning from a Cleaning Route

This section describes how to turn coverage path planning into a lesson requiring no hardware yet highly instructive. It is among the activities this book considers best

suited to a high school classroom, and undergraduates can extend it into a programming exercise.

Why Start from the Cleaning Route

Coverage path planning is a problem everyone has an intuition for and that proves not simple the moment one tries it. Everyone has swept and mopped a floor and knows intuitively to go row by row rather than wander—but defining precisely how to arrange the rows most efficiently, how to go around an obstacle, and how to be sure nothing is missed brings one into the domain of algorithms.

A Paper Activity (suitable for high school)

9. Draw a room on graph paper: a wall for the outer boundary, a few obstacles inside (representing a sofa and a table), and one doorway.
10. Have students plan a route with a pen, required to pass through every cell at least once and to start and end at the doorway, recording the total number of steps.
11. Compare the class's routes: whose took the fewest steps? How much do the shortest and longest differ? Those extra steps are the repetition ratio made concrete.
12. Introduce the boustrophedon strategy: demonstrate cutting the room into several strips, running a boustrophedon within each and connecting them, and compare its efficiency with the intuitive routes.
13. Add variables: if a column stands in the middle of the room, how must the boustrophedon be adjusted? What if the doorway is in a corner rather than the middle?

Observing a Home Cleaning Robot (extended assignment)

14. Observe and record the cleaning route of the robot at home (or in a video): is it random or boustrophedon? How does it handle furniture?
15. Photograph the map in the app and mark which areas are segmented as one room, and whether there is any obvious map error (the symptoms of Section 13.7).
16. Create a challenge deliberately: put a pair of slippers or a (safe) length of cord on the floor and observe whether the robot detects it and how it responds.
17. Pick it up and set it down in another room, and observe the relocalization process (the kidnapping problem of Chapter 3, Case 11-2 of Chapter 11).

Programming Directions for Undergraduates

- Implement boustrophedon coverage planning in Python: take an occupancy grid map with obstacles as input, output a coverage path, and compute coverage and repetition ratio.
- Implement room segmentation: find the doorway bottlenecks with morphological operations or a distance transform and cut the map into blocks.
- In a ROS simulation, connect your own coverage planner to the map built in Chapter 14 or 15 and have a virtual robot perform a complete cleaning.

Part IV, "2D SLAM and the Cleaning Robot," is now complete. We began with the occupancy grid map of Chapter 13, passed through the particle filter of Gmapping in Chapter 14 and the graph optimization and loop closure of Cartographer in Chapter 15, and in this chapter saw how these techniques come down to earth in a home. You should be able to feel that SLAM is not some unreachable frontier technology: its core version is quietly running in your own living room.

In the next part we raise the dimension from the plane to the solid. Part V, "3D

LiDAR SLAM and Autonomous Patrol," introduces three-dimensional point clouds, ICP and NDT registration, the LOAM family of algorithms, and how quadrupeds and patrol robots localize and map in complex three-dimensional space. The difficulty rises, but you have all the necessary groundwork in place.

16.8 The Next Step for Home Cleaning Robots: Four Turns in 2024–2026

The first seven sections of this chapter described a mature combination of technologies: 2D LiDAR mapping, room segmentation, boustrophedon coverage. That combination largely settled after 2017, and the real changes have occurred at its boundaries—once the industry found that the technical bottleneck in cleaning a floor was no longer SLAM but what lies outside SLAM, the direction of innovation turned entirely. This section surveys four such turns.

1. From Going Around Obstacles to Taking Them Away

The most dramatic change came at CES in early 2025: Roborock unveiled the Saros Z70, fitting the cleaning robot with a foldable five-axis arm, OmniGrip, able to pick up small items under 300 grams from the floor—socks, small towels, tissues, slippers—place them where directed, and then come back and clean that patch of floor.

From this book's standpoint, the significance lies not in the arm itself but in the fact that its demands on the map are an entire level higher. To pick up a sock, the robot must know not merely that there is an obstacle there (which an occupancy grid can do) but that it is a sock, that its three-dimensional shape is thus, that it can be gripped, and where it should be put. That goes far beyond the expressive power of the occupancy grid of Chapter 13 and into the territory of semantic maps and object-level

representations. In other words, the real innovation forced out by the arm is an upgrade in the map format.

2. From Geometric Maps to Semantic Maps

Following on from the last point: the mainstream of recent research is to overlay a layer of "what this is" on top of the geometric map. Work on open-vocabulary 3D scene graphs connects the recognition ability of vision-language models to three-dimensional maps, letting a robot respond to tasks described in natural language rather than by coordinates, such as "go to the table beside the sofa."

For a cleaning robot the value of this line is very practical. The room segmentation of Section 16.4 can only cut by geometric bottlenecks and fails at an open kitchen; if the map itself carries object labels, "the area with the counter and the refrigerator is the kitchen" becomes a usable criterion. Keep-out zones are the same—better that the robot recognize the pet's bowl and avoid it than that the user draw a box by hand in the app. The probabilistic framework of Part III handles whether something is there; a semantic map handles what it is, and the two are additive rather than substitutes.

3. The Standardization of Coverage Path Planning: From In-House Algorithms to Open-Source Infrastructure

The third turn is in the software ecosystem. Coverage path planning was for years each manufacturer's secret recipe, but the ROS 2 ecosystem has lately acquired a general coverage planning server (opennav_coverage, built on Fields2Cover) that organizes the strategies of this chapter into configurable options: the path order may be BOUSTROPHEDON, SNAKE, or SPIRAL; the scan angle may be specified or found automatically by brute-force search, and the scoring criterion for "best" is exactly the

three metrics of this chapter—total length, coverage, or number of rows.

This correspondence repays close attention: the evaluation metrics discussed in Section 16.4 become selectable objective functions directly in this open-source tool, and the boustrophedon-versus-spiral comparison the extended challenge asks for is here a configuration parameter. This shows that the content of this chapter is not idle product observation but an engineering field supported by standardized tools. It was designed originally for agricultural machinery (field operations), but the mathematical structure of the coverage problem is identical to indoor cleaning.

4. The Cleaning Robot Chassis as a Standard Platform for Research and Teaching

The last turn concerns you most. As Case 16-3 describes, the cleaning robot chassis is gradually becoming the first choice for a low-cost research platform: the chassis is mature, motor and odometry interfaces are ready-made, the size suits indoor work, the price is far below a research-grade robot, and the flat top makes it easy to add sensor mounts. For university projects and high school clubs on a limited budget, this is a practical route to actually running the theory of sixteen chapters.

This book's advice is: do not chase expensive equipment from the start. A programmable cleaning robot chassis plus one low-cost 2D LiDAR is enough to reproduce everything in Chapters 13 through 16—mapping, loop closure, room segmentation, coverage planning. Put the budget and time saved into the algorithms and field experiments; that is where a project's real value lies.

The Common Signal of the Four Turns

Hardware (the robot arm): cleaning robots begin to act on things rather than

merely go around them.

Representation (semantic maps): the map rises from where something is to what it is, filling the blind spot of purely geometric methods.

Software (open-source coverage planning): coverage path planning goes from each firm's own recipe to a configurable standard component.

Teaching (research chassis): mass-produced consumer hardware flows back as a low-cost platform for research and teaching.

The common signal: SLAM itself is no longer the bottleneck. The technology of the first seven sections remains the foundation, but the competition has moved above that foundation—what more information can be put into the map, and what the robot can do with it.

Looking Back and Forward

Part IV ends here. Looking back at the arrangement of these four chapters: Chapter 13 turned scans into a map, Chapter 14 untied the chicken-and-egg problem with a particle filter, Chapter 15 made history correctable with graph optimization, and Chapter 16 put all three inside a real product and watched how it fares before an 85-centimeter doorway, a 3-centimeter cable, and a mirror. The three sets of experiments and the profile of Figures 16-1 through 16-3 all say the same thing: the elegance of an algorithm is finally accepted or rejected by the geometry of the real world.

From Chapter 17 we raise the dimension from the plane to the solid. Part V, "3D LiDAR SLAM and Autonomous Patrol," introduces three-dimensional point clouds, ICP and NDT registration, the LOAM family of algorithms, and patrol applications. That lonely scan plane in Figure 16-3 is exactly the problem the next part sets out to solve—when

a scan is no longer at a single height, the robot can at last see the whole world.

Formulations and Procedures

1. Coverage and Repetition Ratio

The two core metrics for assessing cleaning efficiency:

$$\textit{coverage} = \textit{area actually cleaned} \;/\; \textit{total cleanable area} \times 100\%$$

$$\textit{repetition ratio} = \textit{actual distance traveled} \;/\; \textit{theoretical shortest covering distance}$$

The theoretical shortest covering distance is roughly the cleanable area divided by the brush width—the distance a perfect boustrophedon would take to cover the whole area without repetition. A repetition ratio of 1 means perfect non-repetition; in practice, with turns, detours around obstacles, and returns to charge, it usually falls between 1.2 and 2.0, while random bump-and-turn may exceed 5.

2. The Number of Rows in a Boustrophedon Path

In an unobstructed rectangle of width W and length L, a boustrophedon at brush width w requires rows and distance:

$$\textit{rows } n = \lceil W \;/\; w \rceil$$

$$\textit{total distance} \approx n \times L + (n - 1) \times w$$

The second term is the lateral movement at each turn. This expression explains why a wider brush is more efficient—fewer rows and fewer turns. It is also the motivation for widening the brush head in product design.

3. The Complete Operating Procedure of a Cleaning Robot

Figure 16-4 The Complete Flow of One Cleaning Run

① Set out from the dock and load the existing map (or enter SLAM mapping mode on the first run)

↓

② Localize: confirm the position in the map by an AMCL-type method (Chapter 11)

↓

③ Room segmentation and task planning: decide the cleaning order and exclude keep-out zones (Section 16.4)

↓

④ Coverage path planning: generate a boustrophedon route within the current room

↓

⑤ Clean: move along the path while

• continuing to localize and update the map (SLAM)

• avoiding obstacles in real time (camera / 3D ToF / bumper / cliff sensors)

• recording the area already cleaned

↓

⑥ Battery low? → plan a return to the dock and, once charged, resume from the point of interruption

↓

⑦ All rooms complete → return to the dock and upload the map and the cleaning record

4. The Basic Approach to Room Segmentation

Figure 16-5 The Distance Transform Method of Room Segmentation

① Run a distance transform on the occupancy grid map: each free cell records its distance to the nearest obstacle

↓

② The center of a room has a large value (open) while doorways and passages have small ones (narrow)

↓

③ Take the regions above a threshold as the core seeds of the rooms

↓

④ Expand outward from each core until they meet → the boundary is the room division

↓

⑤ Output the segmented room blocks for the user to name and assign cleaning individually

Case Studies

Case 16-1 Twenty Years of Evolution: From Roomba to Physical AI

The twenty years of the cleaning robot are a miniature of the popularization of robotics. The first-generation Roomba appeared in 2002 with a random bump-and-turn strategy—no map, no localization, purely the commercialization of the behavior-

based architecture of Chapter 3. In 2010 Neato first introduced 2D LiDAR and SLAM. In 2017 Xiaomi popularized the LiDAR model at an affordable price (Case 1-1 of Chapter 1).

And by 2026 the situation has changed in kind again. This generation of home robots is classified as physical AI—no longer merely rotating brushes steered by random bumps, but running on-device AI that builds maps, avoids obstacles, and adapts to the shape of real domestic life. They scan rooms with LiDAR and camera SLAM and build 2D or 3D maps of walls, furniture, and habitual traffic lines; having built the map, they can label rooms, exclude keep-out zones, and even remember which areas need cleaning more often.

This line of evolution shows clearly that SLAM is the foundation while real products keep layering on sensors and AI to break through the limits of any single technology. From the occupancy grid of Chapter 13 to the semantic labels and behavioral adaptation of this chapter, what you see is the complete thread from Part IV to Part IX (AI semantic navigation) of this book.

Case 16-2 Why Your Cleaning Robot Always Hits That Chair

This is an everyday phenomenon nearly every user has met and the one that best confirms this book's theory. The robot deftly skirts the sofa and the walls yet always drives headlong into the dining chair's legs.

The reason is threefold. First, the angular resolution problem of Chapter 8: a slender chair leg several meters away may fall entirely between two beams and be missed. Second, the height of the scan plane: some chair legs are wide above and narrow below, and the LiDAR happens to scan at the narrowest part. Third, dynamics:

the chair is pulled to a different place every day, and the old position recorded on the map is long out of date.

The remedies correspond to the three levels: rescan at close range (angular resolution improves with distance), add 3D ToF or a camera to see the whole shape, and keep the map plastic so that it reflects the chair's movement (Section 13.5). Next time you see the robot hit a chair leg, consider which chapter of this book each of the three causes corresponds to—the most personal SLAM review there is.

Case 16-3 The Role of the Cleaning Robot Platform in Research and Teaching

Interestingly, the cleaning robot is not only a consumer product but a practical platform for research and teaching. Some research groups have used a programmable cleaning robot chassis as an unmanned ground vehicle, paired it with a low-cost 360-degree 2D LiDAR, and implemented SLAM and coverage path planning research under ROS.

The advantages of such a platform are very practical: the chassis is mature and stable, motor and odometry interfaces are ready-made, the small size suits indoor work, the price is far below a research-grade robot, and the structure readily accepts additional sensor mounts. For a university project or an advanced high school club this is a feasible low-cost route—a consumer chassis with a teaching-grade LiDAR and a single-board computer makes a research platform capable of running the algorithms of Chapters 14 and 15.

This case echoes the discussion of teaching platforms in Section 8.7 of Chapter 8 and offers a lesson: robotics research need not demand expensive equipment, and

making good use of mature consumer hardware as a base while putting one's energy into algorithms and application innovation is often the smarter strategy.

Hands-On Activities

Exercise 16-1 Coverage Path Planning: From Paper to Program

Purpose: to understand the strategies and evaluation metrics of coverage path planning at three levels—on paper, by observation, and in code—and to connect all the theory of Part IV to lived experience.

Part One: Coverage Planning on Paper (everyone, no equipment needed)

1. Draw a 12 × 10 cell room on graph paper with two 2 × 3 obstacles inside (a sofa and a table) and one cell on an edge as the charging dock.
2. Each student plans a route independently, required to pass through every free cell at least once and return to the dock. Record the total steps.
3. Tabulate for the class: what is the fewest number of steps? The most? Compute each student's repetition ratio (total steps ÷ number of free cells).
4. Demonstrate the boustrophedon strategy: cut the room into strips around the obstacles, run a boustrophedon in each, and connect them. Compare the step count with the intuitive routes.
5. Challenge: if the two obstacles are moved side by side into the middle of the room, how must the boustrophedon be adjusted? Draw your solution.

Part Two: Observing a Real Cleaning Robot (homework)

6. Observe the cleaning route of the robot at home (or in an online video), judge whether it is random or planned, and describe how it handles furniture.

7. Screenshot the room map in the app and mark: is the room segmentation reasonable? Are there any of the map symptoms of Section 13.7 (distortion, breakage, drift)?
8. Low obstacle test: put a pair of slippers (a safe object) on the floor and observe whether the robot avoids, pushes, or gets stuck on them, explaining with the knowledge of Section 16.6.
9. Kidnapping test: move the robot to another room mid-cleaning and time how long relocalization takes (the kidnapping problem of Chapter 3, Case 11-2 of Chapter 11).

Part Three: Programming (Undergraduate)

10. Read an occupancy grid map into Python (one produced by the Chapter 13 exercise, or defined yourself) and implement boustrophedon coverage path planning: cut the map into obstacle-free strips, generate a back-and-forth path in each, and connect the strips.
11. Compute and output the coverage and repetition ratio, and visualize the path on the map with matplotlib.
12. Implement room segmentation: use the distance transform method of Figure 16-5 to cut the map into rooms automatically and display them in different colors.
13. Add a keep-out zone feature: let the user specify a rectangular area, exclude it automatically in planning, and verify that the path indeed avoids it.
14. (Advanced) In a ROS simulation, connect your coverage planner to the map built in Chapter 14 or 15 and have a virtual robot perform a complete cleaning.

Discussion Questions

1. In Part One, how many steps did the boustrophedon route save over the intuitive routes? In a real 100-ping home, how much cleaning time would that difference amount to?
2. What map symptoms did you find while observing a real robot? Judging by Section 13.7, what are their most likely causes?
3. How long did relocalization take in the kidnapping test? If the robot were placed in a room that looks like its original location (two bedrooms of identical layout, say), what would you expect to happen? (Hint: the multimodal distribution of Chapter 11.)
4. Following Table 16-3, if you were designing a cleaning robot for a home with pets, which sensors would you strengthen first, and why?

Extended Challenge (Undergraduate)

First: compare two coverage strategies—boustrophedon and spiral (circling inward from the outside)—on the same map for coverage, repetition ratio, and number of turns, and analyze what shape of room each suits. Second: implement resuming after a break—interrupt the path halfway, record the completed area, and on restart clean only the remainder, verifying that coverage still reaches 100%. This feature requires localization, the map, and task state management at once, and is a good topic for integrating everything in Part IV.

Further Reading

1. H. Choset, "Coverage for Robotics – A Survey of Recent Results," Annals of Mathematics and Artificial Intelligence, 2001. (The classic survey of coverage path planning, including boustrophedon decomposition.)

2. E. Galceran and M. Carreras, "A Survey on Coverage Path Planning for Robotics," Robotics and Autonomous Systems, 2013. (A complete review of coverage planning, suitable as a project reference.)
3. R. Bormann et al., "Room Segmentation: Survey, Implementation, and Analysis," ICRA, 2016. (A comparative study of room segmentation methods, with open-source implementations.)
4. S. Thrun, W. Burgard, and D. Fox, Probabilistic Robotics, MIT Press, 2005. (The theoretical basis of occupancy grids and Monte Carlo localization, echoing this chapter's mapping and relocalization.)
5. R. Chuchro, "Simulating Coverage Path Planning with Roomba," arXiv, 2022. (Explores coverage planning with reinforcement learning and compares it with a cleaning robot's built-in algorithm.)
6. The Coverage Server documentation of ROS Navigation2 (docs.nav2.org). (The coverage path planning tool of the ROS 2 ecosystem.)
7. The official OpenCV tutorials on the distance transform and morphological operations. (The technical basis for implementing room segmentation.)

Summary

1. A cleaning robot needs SLAM to answer what has been cleaned, when it is finished, and how to get home; SLAM brings four capabilities—systematic cleaning, resuming after a break, user interaction, and consistency across cleanings—and is a physical review of the first fifteen chapters of this book.
2. Random bump-and-turn is the commercialization of reactive architecture (the 2002 Roomba), with a disorderly route and no guarantee of coverage; SLAM-based planning maps first and then runs a boustrophedon path, with high

coverage and a low repetition ratio, though a map error leads directly to a cleaning error.

3. A LiDAR cleaning robot builds an occupancy grid map with a low-cost 2D LiDAR, running SLAM on the first cleaning and loading the map to localize thereafter; modern premium models use LiDAR and an AI camera together—LiDAR to map and localize, the camera to recognize objects on the floor—the commercial practice of "LiDAR gives geometry, the camera gives semantics."
4. Room segmentation cuts the map into blocks by finding spatial bottlenecks such as doorways and already touches on the semantic map idea; a keep-out zone injects the user's knowledge into the costmap; coverage path planning seeks to visit every cell rather than reach a point, and is assessed by the three metrics of coverage, repetition ratio, and cleaning time.
5. The sensor suite comprises a 2D LiDAR, an IMU, wheel encoders, bumper, cliff, infrared or ultrasonic sensors, an AI camera, and 3D ToF; the reason for such variety is the 2D LiDAR's height blind spot—low obstacles, stairs and drop-offs, and glass and mirrors all require other sensors to fill the gap.
6. The five challenges of a home: slender chair and table legs (insufficient angular resolution), mirrors and floor-to-ceiling windows (the laser passing through or reflecting), low and dynamic obstacles (below the scan plane), an environment that changes daily (the map must both remember and forget), and changes in floor material (degraded odometry).
7. Coverage path planning is excellent teaching material—everyone has an intuition for it and it proves not simple the moment one tries—and can be taught at three levels, from planning on paper through observing a real robot to programming, making the best bridge between theory and daily life.

Part IV, "2D SLAM and the Cleaning Robot," ends here. We began with the occupancy grid map, passed through the particle filter of Gmapping and the graph optimization and loop closure of Cartographer, and finally saw how these techniques come down to earth in a home—SLAM is not some unreachable frontier technology; it is quietly running in your own living room. In the next part we raise the dimension from the plane to the solid: Part V, "3D LiDAR SLAM and Autonomous Patrol," introduces three-dimensional point cloud processing, ICP and NDT registration, the LOAM family of algorithms, and how quadrupeds and patrol robots localize and map in complex three-dimensional space.